

5

Session

Pentesting Android Applications

Thực hành Bảo mật web và ứng dụng



Lưu hành nội bộ 2026

(Ng nghiêm cấm đăng tải trên internet dưới mọi hình thức)

A. TỔNG QUAN

A.1 Mục tiêu

Giúp sinh viên có kiến thức và kỹ năng cơ bản trong kiểm thử một ứng dụng Android đơn giản.

B. CHUẨN BỊ MÔI TRƯỜNG

B.1.1 Các tập tin được cung cấp sẵn

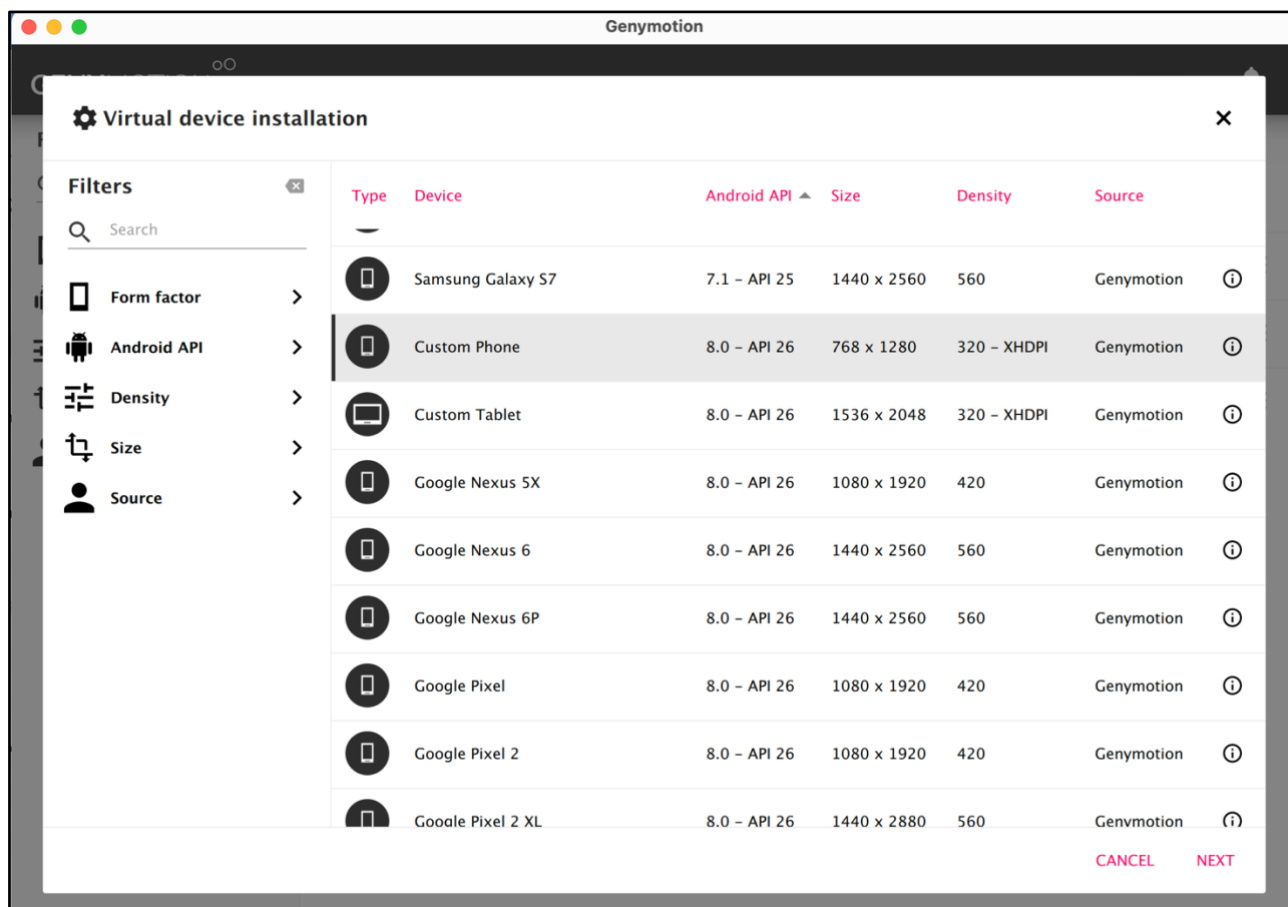
Với mỗi yêu cầu bên dưới, sinh viên tiến hành phân tích một số file APK được giảng viên cung cấp sẵn, bao gồm:

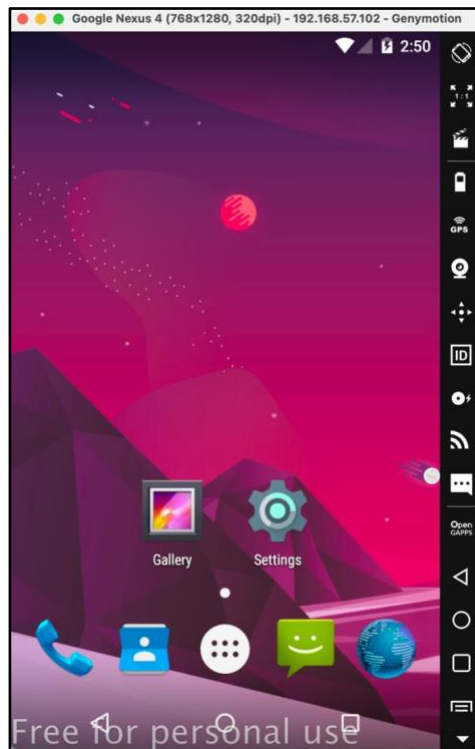
- **InsecureBankv2.apk**: file ứng dụng InsecureBank đã có tùy chỉnh, dùng cho yêu cầu phân tích file APK.
- **AndroLabServer**: thư mục chứa các file Python dùng để chạy server web cho ứng dụng, dùng python3.

B.1.2 Điện thoại ảo

Ta cần hai phần mềm:

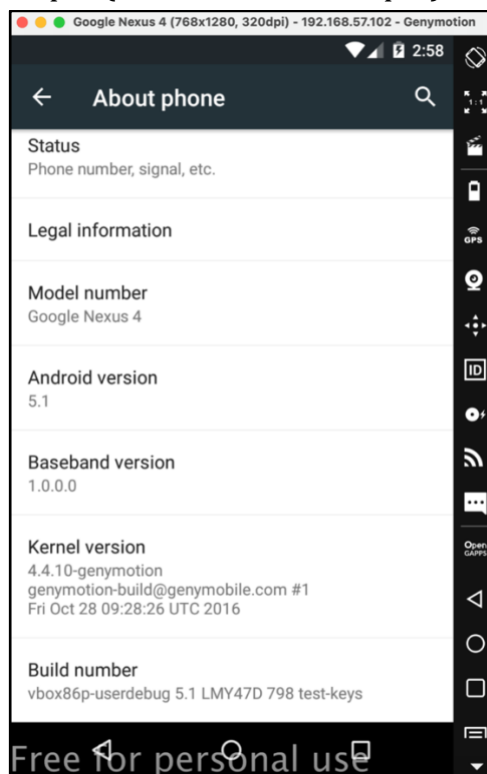
- Virtualbox (<https://www.virtualbox.org/wiki/Downloads>): để chạy Genymotion.
- Genymotion (<https://www.genymotion.com/download/>): Virtualbox dùng làm nền cho Genymotion chạy máy ảo.
- Dùng Genymotion tạo điện thoại ảo Non-root, tham khảo danh sách các phiên bản không root tại: <https://support.genymotion.com/hc/en-us/articles/360003125397-Is-it-possible-to-un-root-or-hide-root>



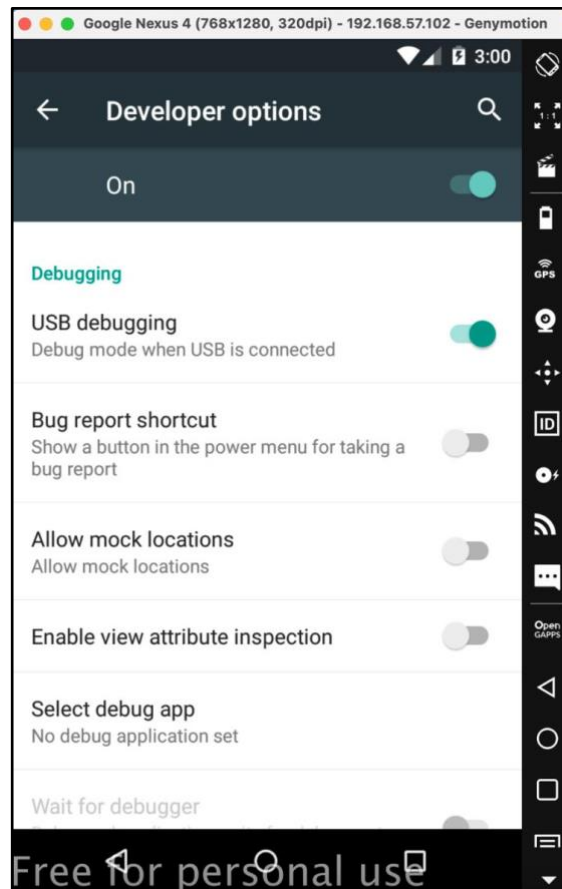


B.1.3 Công cụ Android

- Cài đặt Android Debug Bridge (ADB):
 - Ubuntu/Kali (Linux): `sudo apt install android-tools-adb`
 - MacOS: `brew install android-platform-tools`
 - Windows: theo hướng dẫn [tại đây](#)
- Trên điện thoại ảo vừa tạo, chọn Settings > About Phone > Bấm 6 đến 12 lần vào Build number để bật chế độ developer (You are now developer)



- Về lại phần Settings, sẽ thấy mục Developer options xuất hiện. Chọn Developer options > USB Debugging lên, chức năng này cho phép tương tác giữa điện thoại ảo và máy thật.



- Bây giờ có thể cài apk bằng adb

```
~/Desktop/Android
adb devices
List of devices attached
192.168.57.102:5555    device
```

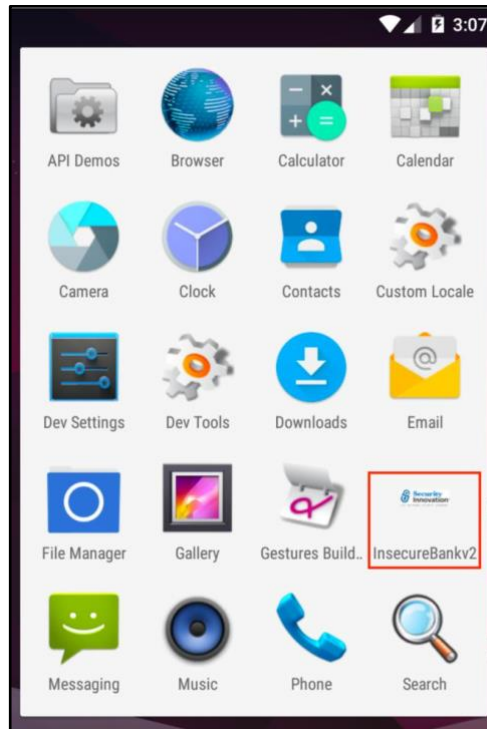
192.168.57.102 là địa chỉ ip của điện thoại ảo và sẽ tương tác từ máy thật qua port 5555.

- Cài đặt apk vào điện thoại bằng câu lệnh.

```
adb install InsecureBankv2.apk
```

```
~/Desktop/Android
adb install InsecureBankv2.apk
Performing Push Install
InsecureBankv2.apk: 1 file pushed, 0 skipped. 26.2 MB/s (3462429 bytes in 0.126s)
    pkg: /data/local/tmp/InsecureBankv2.apk
Success
```


- Xem trên điện thoại, thấy tập tin đã cài đặt thành công



- Để vào terminal shell của điện thoại, gõ lệnh:

```
adb shell
```

```

root@vbox86p:/ # whoami
root
root@vbox86p:/ # uname -a
Linux localhost 4.4.10-genymotion #1 SMP PREEMPT Fri Oct 28 09:28:26 UTC 2016 x86_64 GNU/Linux
root@vbox86p:/ #

```

- Để xem log, gõ lệnh:

```

adb logcat
V/BackupManagerService( 728): restoreAtInstall pkg=com.android.insecurebankv2 token=3 restoreSet=0
V/BackupManagerService( 728): Finishing install immediately
W/Settings( 728): Setting install_non_market_apps has moved from android.provider.Settings.Global to
android.provider.Settings.Secure, returning read-only value.
I/InputReader( 728): Reconfiguring input devices. changes=0x00000010
D/BackupManagerService( 728): Received broadcast Intent { act=android.intent.action.PACKAGE_ADDED dat
=package:com.android.insecurebankv2 flg=0x4000010 (has extras) }
I/ActivityManager( 728): Start proc 2406:com.svox.pico/u0a49 for broadcast com.svox.pico/.VoiceDataIn
stallerReceiver
W/VoiceInteractionManagerService( 728): no available voice recognition services found for user 0
E/libprocessgroup( 2406): failed to make and chown /acct/uid_10049: Read-only file system
W/Zygote ( 2406): createProcessGroup failed, kernel missing CONFIG_CGROUP_CPUACCT?
I/art ( 728): Explicit concurrent mark sweep GC freed 24189(1531KB) AllocSpace objects, 10(241KB)
LOS objects, 33% free, 7MB/11MB, paused 1.948ms total 68.344ms
D/Launcher.Workspace( 1622): 11683562 - removeExtraEmptyScreen()
D/Launcher.Workspace( 1622): 11683562 - convertFinalScreenToEmptyScreenIfNecessary()
I/art ( 2383): System.exit called, status: 0
I/AndroidRuntime( 2383): VM exiting with result code 0.
I/ActivityManager( 728): Killing 1497:android.process.media/u0a5 (adj 15): empty #17
W/libprocessgroup( 728): failed to open /acct/uid_10005/pid_1497/cgroup.procs: No such file or direct
ory
I/InputReader( 728): Reconfiguring input devices. changes=0x00000010
D/BackupManagerService( 728): Received broadcast Intent { act=android.intent.action.PACKAGE_CHANGED d
at=package:com.android.musicfx flg=0x4000010 (has extras) }
W/Launcher( 1622): setApplicationContext called twice! old=com.android.launcher3.LauncherApplication@2
2c5d2af new=com.android.launcher3.LauncherApplication@22c5d2af
I/art ( 1374): Background sticky concurrent mark sweep GC freed 5942(397KB) AllocSpace objects, 0(
0B) LOS objects, 19% free, 2MB/2MB, paused 16.120ms total 24.745ms
I/UsageStatsService( 728): User[0] Flushing usage stats to disk

```

- Để tải lên một tập tin từ máy thật đến điện thoại ảo, gõ lệnh:

```
adb push <path máy thật> <path máy ảo>
```

```
❏ ~/Desktop/Android
echo "test" > test

❏ ~/Desktop/Android
adb push test /tmp/
test: 1 file pushed, 0 skipped. 0.0 MB/s (5 bytes in 0.003s)

❏ ~/Desktop/Android
adb shell
root@vbox86p:/ # cat /tmp/test
test
root@vbox86p:/ #
```

- Để tải một tập tin từ điện thoại ảo về máy thật, gõ lệnh:

```
root@vbox86p:/ # echo "device" > /tmp/device
root@vbox86p:/ # exit

❏ ~/Desktop/Android
adb pull /tmp/device ./
/tmp/device: 1 file pulled, 0 skipped. 0.0 MB/s (7 bytes in 0.002s)

❏ ~/Desktop/Android
cat device
device
```

C. THỰC HÀNH

C.1 Phân tích tĩnh (Static Analysis)

Phân tích tĩnh là quá trình rà soát mã nguồn, cấu hình để xem xét lỗ hổng. Song song với việc làm thủ công có thể kết hợp với công cụ tự động để giúp kết quả đạt độ chính xác cao hơn.

Mobile Security Framework (MobSF) là công cụ kiểm thử di động tự động (Android/iOS/Windows), phân tích mã độc, hỗ trợ phân tích tĩnh lẫn phân tích động. MobSF hỗ trợ tập tin binaries (APK, IPA & APPX).



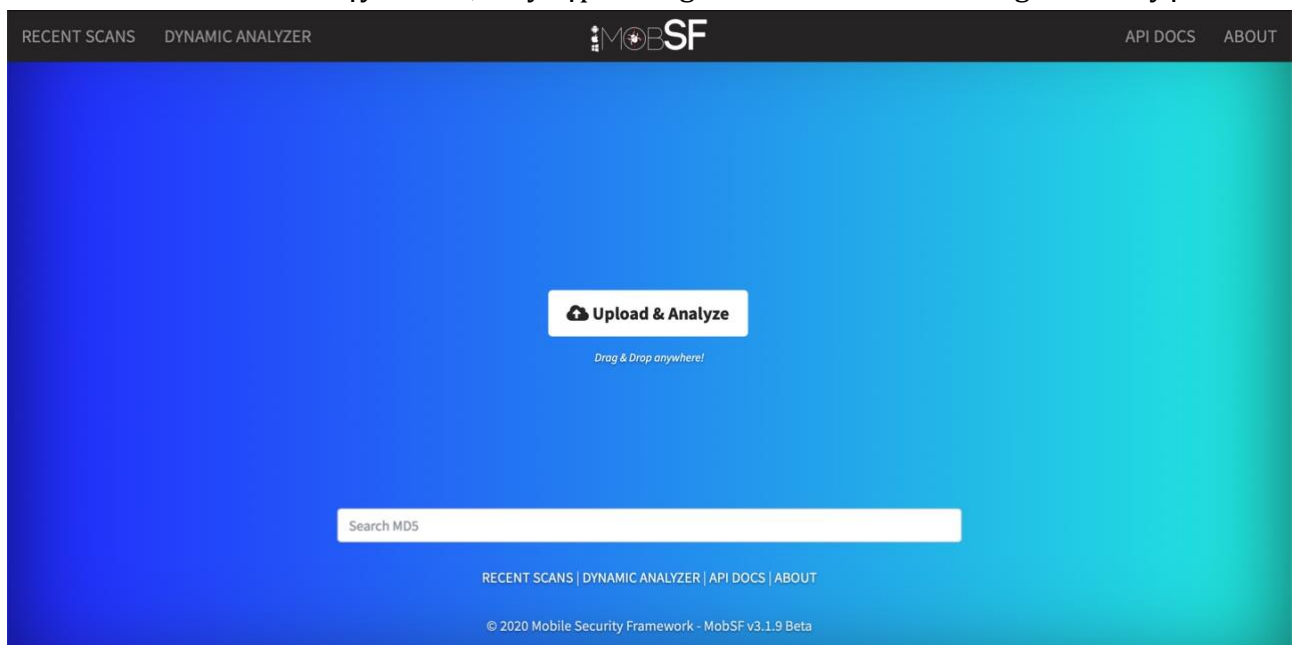
C.1.1 Cài đặt

Bước 1. Xem chuẩn bị môi trường và cài đặt tại: <https://mobsf.github.io/docs/#/>.

Bước 2. Bật MobSF bằng lệnh:

```
./run.sh
```

Bước 3. Sau khi chạy server, truy cập đường dẫn `localhost:8000` bằng trình duyệt.



C.1.2 Phân tích tự động

Tải lên hoặc kéo thả tập tin *InsecureBankv2.apk* MobSF để tiến hành phân tích tĩnh.

The screenshot shows the MobSF Static Analyzer interface. The left sidebar contains navigation options: Information, Scan Options, Signer Certificate, Permissions, Android API, Browsable Activities, Security Analysis, Malware Analysis, Reconnaissance, Components, PDF Report, Print Report, and Start Dynamic Analysis. The main panel displays the following information:

- APP SCORES:** Average CVSS 0, Security Score 100/100, Trackers Detection 3/319.
- FILE INFORMATION:** File Name: InsecureBankv2.apk, Size: 3.3MB, MD5: See4829065640f9c936ac861d1650ffc, SHA1: 80b53f80a3c9e6bfd98311f5b26ccddcd1bf0a98, SHA256: b18af2a0e44d7634bbcdf93664d9c78a2695e050393fcfb5e8b91f902d194a4.
- APP INFORMATION:** App Name: InsecureBankv2, Package Name: com.android.insecurebankv2, Main Activity: com.android.insecurebankv2.LoginActivity, Target SDK: 22, Min SDK: 15, Max SDK: 22, Android Version Name: 1.0, Android Version Code: 1.
- Component Counts:**
 - 10 ACTIVITIES (Exported Activities: 4)
 - 0 SERVICES (Exported Services: 0)
 - 2 RECEIVERS (Exported Receivers: 1)
 - 1 PROVIDERS (Exported Providers: 1)
- SCAN OPTIONS:** Buttons for Scan, Stop, and Refresh.
- DECOMPILED CODE:** Buttons for Download, View, and Copy.

Kiểm tra các cấu hình phân quyền

The screenshot shows the MobSF Static Analyzer interface with the 'APPLICATION PERMISSIONS' section selected. The left sidebar is the same as in the previous screenshot. The main panel displays a table of permissions:

PERMISSION	STATUS	INFO	DESCRIPTION
android.permission.ACCESS_COARSE_LOCATION	dangerous	coarse (network-based) location	Access coarse location sources, such as the mobile network database, to determine an approximate phone location, where available. Malicious applications can use this to determine approximately where you are.
android.permission.ACCESS_NETWORK_STATE	normal	view network status	Allows an application to view the status of all networks.
android.permission.GET_ACCOUNTS	dangerous	list accounts	Allows access to the list of accounts in the Accounts Service.
android.permission.INTERNET	normal	full Internet access	Allows an application to create network sockets.
android.permission.READ_CONTACTS	dangerous	read contact data	Allows an application to read all of the contact (address) data stored on your phone. Malicious applications can use this to send your data to other people.
android.permission.READ_PROFILE	dangerous	read the user's personal profile data	Allows an application to read the user's personal profile data.
android.permission.SEND_SMS	dangerous	send SMS	Allows application to send SMS messages. Malicious applications may use this to send unsolicited text messages to other users.

Kiểm tra cấu hình trong tập tin Manifest

MANIFEST ANALYSIS			
Search:			
NO ↑↓	ISSUE ↑↓	SEVERITY ↑↓	DESCRIPTION ↑↓
1	Debug Enabled For App [android:debuggable=true]	high	Debugging was enabled on the app which makes it easier for reverse engineers to hook a debugger to it. This allows dumping a stack trace and accessing debugging helper classes.
2	Application Data can be Backed up [android:allowBackup=true]	medium	This flag allows anyone to backup your application data via adb. It allows users who have enabled USB debugging to copy application data off of the device.
3	Activity (com.android.insecurebankv2.PostLogin) is not Protected. [android:exported=true]	high	An Activity is found to be shared with other apps on the device therefore leaving it accessible to any other application on the device.
4	Activity (com.android.insecurebankv2.DoTransfer) is not Protected. [android:exported=true]	high	An Activity is found to be shared with other apps on the device therefore leaving it accessible to any other application on the device.
5	Activity (com.android.insecurebankv2.ViewStatement) is not Protected. [android:exported=true]	high	An Activity is found to be shared with other apps on the device therefore leaving it accessible to any other application on the device.
6	Content Provider (com.android.insecurebankv2.TrackUserContentProvider)	high	A Content Provider is found to be shared with other apps on the device therefore leaving it accessible to any other application on the device.

MobSF cung cấp thông cần thiết để đánh giá độ bảo mật của chương trình

SIGNER CERTIFICATE	
APK is signed v1 signature: True v2 signature: False v3 signature: False Found 1 unique certificates Subject: ST=MA, L=Boston, O=SI, OU=Services, CN=Dinesh Shetty Signature Algorithm: rsassa_pkcs1v15 Valid From: 2015-07-24 20:37:08+00:00 Valid To: 2040-07-17 20:37:08+00:00 Issuer: ST=MA, L=Boston, O=SI, OU=Services, CN=Dinesh Shetty Serial Number: 0x6bb4f616 Hash Algorithm: sha256 md5: 6a736d89abb13d7165e7c9f905ac928d sha1: a1bae91a2b1620f6c9dab425e69fc32ba1e97741 sha256: 8092db81ae717486631a1534977def465ee112903e1553d38d41df8abd57a375 sha512: 53770f3f69916f74ddd6e750ae16fd9b23fa5b2c8e9e53bd5a84202d7d7c44a26ede13e6db450ab0c1d9f64534802b88ebb0b4de1da076b62112d9b122cbbd92	
Search:	
STATUS ↑↓	DESCRIPTION ↑↓
bad	Application is signed with v1 signature scheme, making it vulnerable to Janus vulnerability on Android <7.0
secure	Application is signed with a code signing certificate

A STRINGS

```

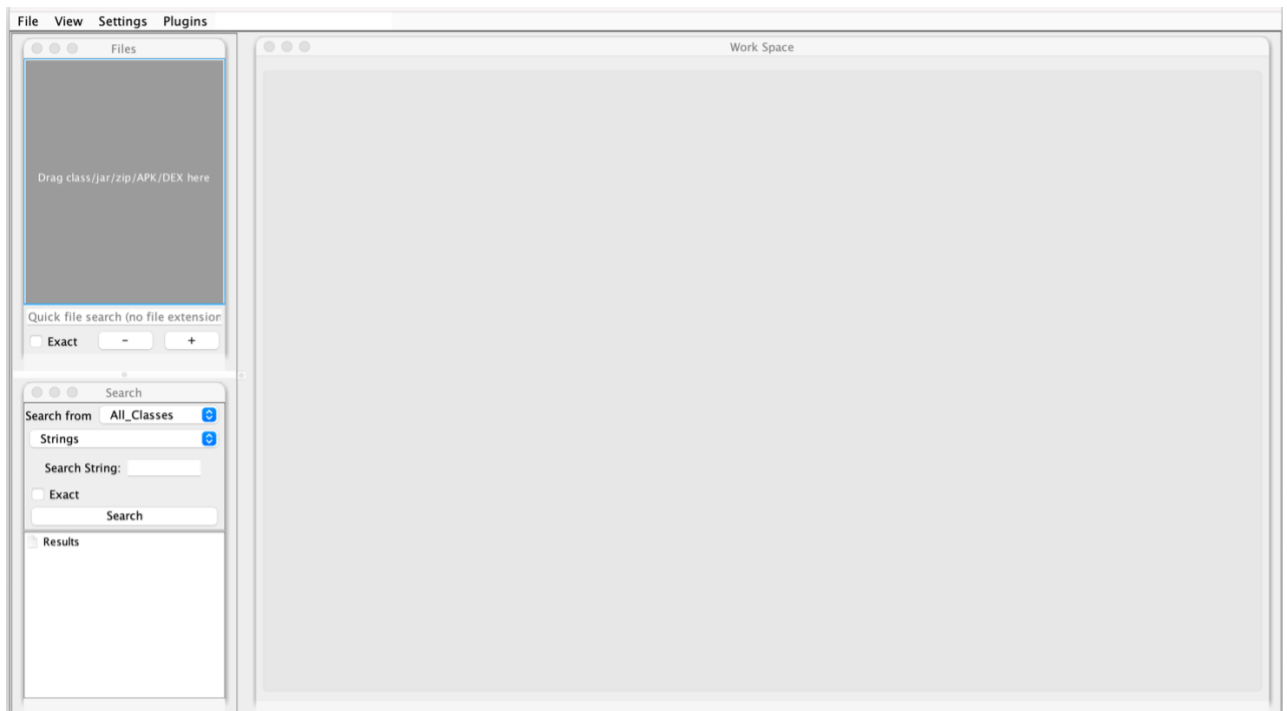
"abc_action_bar_up_description": "اوپر نیویگیٹ کریں"
"common_google_play_services_update_title": "Opdater Google Play-tjenester"
"common_android_wear_update_title": "Eguneratu Android Wear"
"mr_media_route_controller_stop": "Заустави пребацивање"
"common_google_play_services_api_unavailable_text": "%1$s memerlukan satu atau lebih perkhidmatan Google Play yang tidak tersedia pada masa ini. Sila hubungi pembangun untuk mendapatkan bantuan."
"common_google_play_services_unknown_issue": "ມີຂໍ້ບຶກຂ້າງຄົດໃນບໍລິການ Google Play."
"common_google_play_services_update_text": "Lai lietotne %1$s darbotos, jums ir jāatjaunina Google Play pakalpojumi."
"abc_action_bar_up_description": "Navigate up"
"mr_user_route_category_name": "Уреди"
"common_google_play_services_install_text_phone": "您的手机没有安装 Google Play 服务，因此无法运行%1$s。"
"common_google_play_services_update_title": "Ažuriraj usluge za Google Play"
"abc_searchview_description_submit": "Indsend forespørgslen"
"common_google_play_services_needs_enabling_title": "ఒక అనువర్తనానికి Google Play సేవలు ప్రారంభించబడాలి."
"abc_action_mode_done": "Kész"
"abc_activity_chooser_view_see_all": "ಎಲ್ಲವನ್ನೂ ನೋಡಿ"
"common_android_wear_update_text": "%1$s no se ejecutará hasta que no actualices la aplicación Android Wear."
"abc_action_bar_home_description": "הגה בית"
"mr_media_route_chooser_searching": "உவகூடுகளாகக் காணவில்லை"
"mr_media_route_controller_stop": "Löpetä ülekanne"
"common_google_play_services_sign_in_failed_title": "साइन इन असफल भयो"
"common_google_play_services_unknown_issue": "Google சேர் பரிசீலனா செய்ய வேண்டியது"
"common_google_play_services_enable_title": "Activati Servicii Google Play"
"mr_media_route_controller_pause": "Паўзаванне"
"mr_media_route_controller_play": "Phát"

```

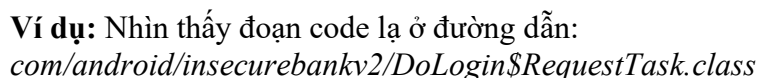
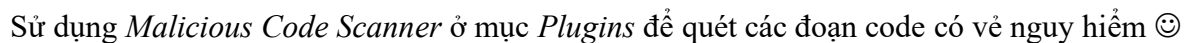
C.1.3 Phân tích thủ công

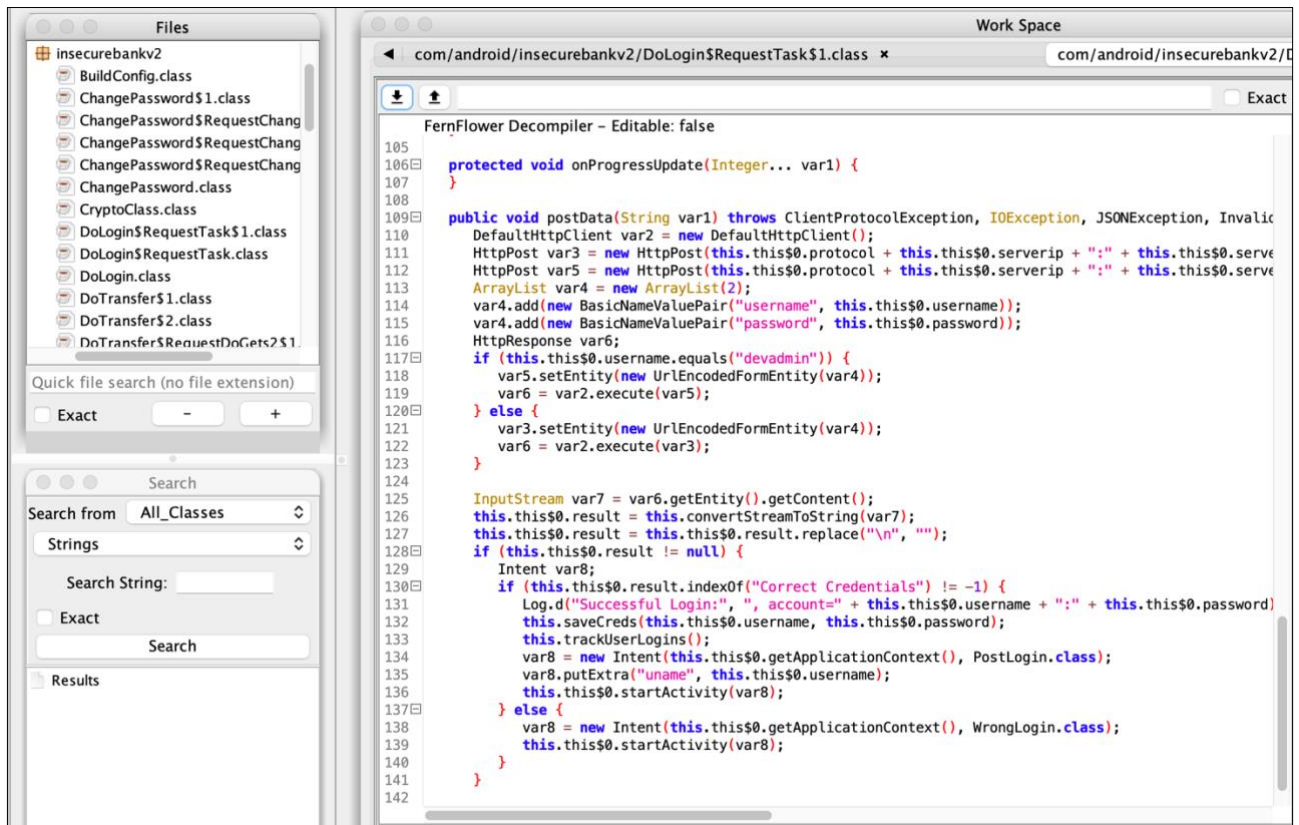
Một trong các bước quan trọng trong phân tích tĩnh là kiểm tra code. Để làm được việc này dùng ByteCode Viewer để xem code nhanh nha trong tập tin apk. Tải tập tin **Bytecode-Viewer-2.9.x.jar** từ <https://github.com/Konloch/bytecode-viewer/releases> và mở chương trình bằng dòng lệnh sau:

```
java -jar Bytecode-Viewer-2.9.x.jar
```



Kéo thả tập tin apk, ByteCode Viewer sẽ phân tích và hiển thị code.





Yêu cầu 1 Phân tích và chỉ ra điểm bất thường của đoạn code trên?

Thông tin đăng nhập không được mã hoá trong cơ sở dữ liệu

Cài đặt chương trình *InsecureBankv2* lên máy ảo Android, thử đăng nhập để chương trình lưu thông tin (**dinesh/Dinesh@123\$** hoặc **jack/Jack@123\$**).

Từ máy chính, gõ **adb shell** để vào command line của máy ảo Android.

```
generic_x86_arm:/data/data/com.android.insecurebankv2/databases # ls
mydb mydb-journal
generic_x86_arm:/data/data/com.android.insecurebankv2/databases # sqlite3 mydb
SQLite version 3.28.0 2020-05-06 18:46:38
Enter ".help" for usage hints.
sqlite>
```

Yêu cầu 2 Chỉ ra rằng dữ liệu lưu trữ có an toàn hay không?

Thông tin nhạy cảm lưu dưới dạng plain-text

- Sử dụng chương trình
- Vào **adb shell** đi đến **/data/data/com.android.insecurebankv2**
- Chạy câu trúc lệnh sau:

```
grep -r <string-to-find> $(find)
```

Yêu cầu 3 Kiểm tra xem thông tin nhạy cảm có lưu lại trên thiết bị hay không? Một số từ khoá: deviceId, userId, imei, deviceSerialNumber, devicePrint, phone, XDSN, mdn, IMSI, uuid...

Tập tin sao lưu không được mã hoá

Nếu app cho phép backup, ta sẽ thử tính năng này xem chương trình ứng dụng lưu những gì.

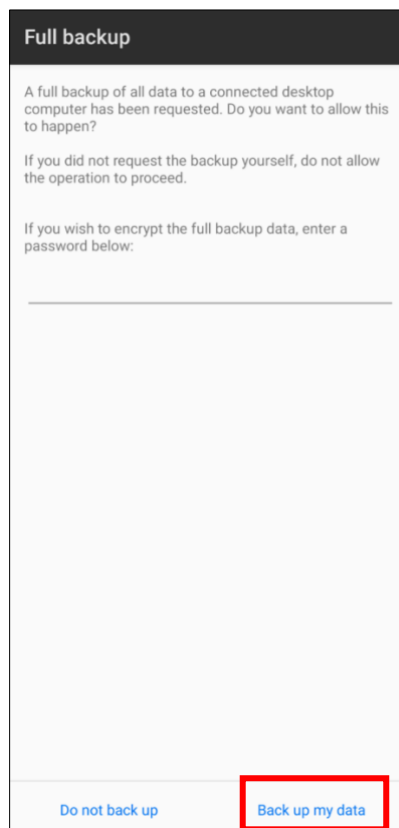
```
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>
<uses-feature android:glEsVersion="20000" android:required="true"/>
<application android:theme="@style/Theme.Holo.Light.DarkActionBar" android:label="@string/app_name"
    android:allowBackup="true">
    <activity android:label="@string/app_name" android:name="com.android.insecurebankv2.LoginActivit
        <intent-filter>
            <action android:name="android.intent.action.MAIN"/>
            <category android:name="android.intent.category.LAUNCHER"/>
        </intent-filter>
    </activity>
    <activity android:label="@string/title_activity_file_pref" android:name="com.android.insecurebar
        android:adjustUnspecified|stateVisible|adjustResize|adjustPan|adjustNothing"/>
```

1. Đăng nhập với tài khoản người dùng bình thường (**dinesh/Dinesh@123\$** hoặc **jack/Jack@123\$**)
2. Thực thi lệnh sau để sao lưu:

```
adb backup -apk -shared com.android.insecurebankv2
```

```
> adb backup -apk -shared com.android.insecurebankv2
WARNING: adb backup is deprecated and may be removed in a future release
Now unlock your device and confirm the backup operation...
```

3. Tại giao diện điện thoại tiến hành cho phép backup



4. Sao lưu được tạo

```
> ls | grep backup.ab
backup.ab
```

5. Chuyển đổi tập tin sao lưu qua định dạng có thể đọc được (cài gói qpdf)

```
cat backup.ab | (dd bs=24 count=0 skip=1; cat) | zlib-flate -uncompress >
backup_compressed.tar
```

```
> cat backup.ab | (dd bs=24 count=0 skip=1; cat) | zlib-flate -uncompress > backup_compressed.tar
0+0 records in
0+0 records out
0 bytes transferred in 0.000014 secs (0 bytes/sec)
> tar -zxvf backup_compressed.tar
x apps/com.android.insecurebankv2/_manifest
x apps/com.android.insecurebankv2/a/base.apk
x apps/com.android.insecurebankv2/db/mydb-journal
x apps/com.android.insecurebankv2/db/mydb
x apps/com.android.insecurebankv2/sp/mySharedPreferences.xml
x apps/com.android.insecurebankv2/sp/com.android.insecurebankv2_preferences.xml
x shared/0/Documents
x shared/0/Audiobooks
x shared/0/DCIM
x shared/0/Download
x shared/0/Movies
x shared/0/Movies/.thumbnails
x shared/0/Movies/.thumbnails/.nomedia
x shared/0/Movies/.thumbnails/.database_uuid
x shared/0/Pictures
x shared/0/Pictures/.thumbnails
x shared/0/Pictures/.thumbnails/.nomedia
x shared/0/Pictures/.thumbnails/.database_uuid
x shared/0/Notifications
x shared/0/Alarms
```

6. Kiểm tra các tập tin sao lưu sau khi giải nén

```
> cd apps/com.android.insecurebankv2/sp
> ls -la
total 16
drwxr-xr-x  4 phaphajian  staff  128 Nov 24 13:39 .
drwxr-xr-x  6 phaphajian  staff  192 Nov 24 13:39 ..
-rw-r-----  1 phaphajian  staff  160 Nov 23 16:09 com.android.insecurebankv2_preferences.xml
-rw-r-----  1 phaphajian  staff  221 Nov 23 16:10 mySharedPreferences.xml
> cat mySharedPreferences.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
  <string name="superSecurePassword">v/sJpihDCo2ckDmLW5Uwiw==&#10;    </string>
  <string name="EncryptedUsername">amFjaw==&#13;&#10;    </string>
</map>
```

Yêu cầu 4 Theo bạn thư mục sao lưu chứa thông tin nào cần mã hoá, chỉ ra.

Phương thức mã hoá yếu

Tại đường dẫn `com/android/insecurebankv2/CryptoClass.class`, chứa các hàm mã hoá và giả mã.

```

14
15 public class CryptoClass {
16     String base64Text;
17     byte[] cipherData;
18     String cipherText;
19     byte[] ivBytes = new byte[]{0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0};
20     String key = "This is the super secret key 123";
21     String plaintext;
22
23 public static byte[] aes256decrypt(byte[] var0, byte[] var1, byte[] var2) throws UnsupportedOperationException, NoSuchAlgorithmException {
24     IvParameterSpec var4 = new IvParameterSpec(var0);
25     SecretKeySpec var5 = new SecretKeySpec(var1, "AES");
26     Cipher var3 = Cipher.getInstance("AES/CBC/PKCS5Padding");
27     var3.init(2, var5, var4);
28     return var3.doFinal(var2);
29 }
30
31 public static byte[] aes256encrypt(byte[] var0, byte[] var1, byte[] var2) throws UnsupportedOperationException, NoSuchAlgorithmException {
32     IvParameterSpec var4 = new IvParameterSpec(var0);
33     SecretKeySpec var3 = new SecretKeySpec(var1, "AES");
34     Cipher var5 = Cipher.getInstance("AES/CBC/PKCS5Padding");
35     var5.init(1, var3, var4);
36     return var5.doFinal(var2);
37 }
38
39 public String aesDecryptedString(String var1) throws UnsupportedOperationException, InvalidKeyException, NoSuchAlgorithmException {
40     byte[] var2 = this.key.getBytes("UTF-8");
41     this.cipherData = aes256decrypt(this.ivBytes, var2, Base64.decode(var1.getBytes("UTF-8"), 0));

```

- Ta có key: "This is the super secret key 123"
- Ta có đoạn dữ liệu bị mã hoá

```

> cat mySharedPreferences.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
    <string name="superSecurePassword">v/sJpihDCo2ckDmLW5Uwiw==&#10;    </string>
    <string name="EncryptedUsername">amFjaw==&#13;&#10;    </string>
</map>

```

Yêu cầu 5 Viết chương trình giải mã đoạn dữ liệu mã hoá (python3 chẳng hạn...)

```

~/Desktop/Android 1 x 20:58:50
python3 -m pip install pycrypto
Defaulting to user installation because normal site-packages is not writeable
Collecting pycrypto
  Using cached pycrypto-2.6.1.tar.gz (446 kB)
Building wheels for collected packages: pycrypto
  Building wheel for pycrypto (setup.py) ... done
  Created wheel for pycrypto: filename=pycrypto-2.6.1-cp38-cp38-macosx_10_14_6_x86_64.whl size=606112 sha256=976b39567eac9886338a359c556bfeae422f83ebf4bb7dd9937dff786e5c8f9a
  Stored in directory: /Users/phaphajian/Library/Caches/pip/wheels/d0/99/d0/029ea019d63f1d63a0965b9944b719e875f9bd6ffc6dcf293
Successfully built pycrypto
Installing collected packages: pycrypto
Successfully installed pycrypto-2.6.1

~/Desktop/Android ✓ 32s 20:59:31
python3 decrypt.py
Jack@123$

```

Activity Hijacking

- Gọi Activity Manager (am): là lệnh gọi các lệnh của hệ thống; ví dụ chạy Activity, dừng một tiến trình, sửa đổi các thuộc tính...
- Thử gọi activity PostLogin, dùng lệnh sau:

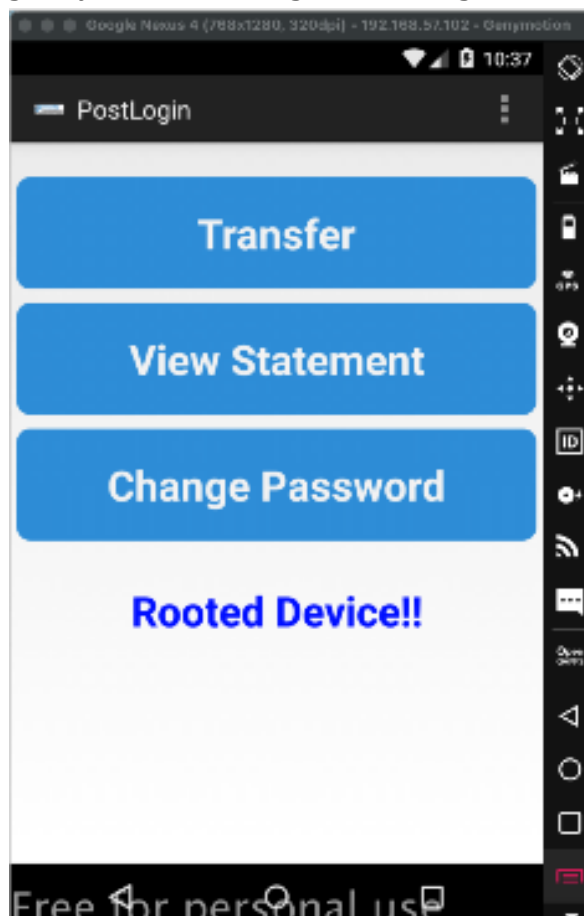
```
am start -n com.android.insecurebankv2/.PostLogin
```

```

~ / Desktop / Android
adb shell
root@vbox86p:/ # am start -n com.android.insecurebankv2/.PostLogin
Starting: Intent { cmp=com.android.insecurebankv2/.PostLogin }
root@vbox86p:/ #

```

App tự động chuyển đến PostLogin mà không cần đăng nhập.



C.2 Dịch ngược (Reverse Engineering)

C.2.1 Bên trong APK

- APK (Android application package) là 1 tập tin chứa các thành phần cần thiết cho việc cài đặt ứng dụng trên thiết bị Android.
- Giống như Windows có tập tin .exe Linux có tập tin .elf, thì Android cũng có tập.apk, quá trình cài đặt tương tự.
- Việc cài đặt thủ công (cài bằng tập tin apk) được gọi là side-loading.
- Dưới đây là các thành phần thường có trong tập tin apk:
 - META-INF/: Thư mục này có ở trong tập apk đã ký, nó bao gồm các danh sách tập tin trong APK và chữ kí của chúng.

- lib/: Các file của thư viện native (file có đuôi *.so) được lưu vào thư mục con của lib/ như x86, x86_64.
- res/: thư mục này gồm tất cả các resource XML, drawables (PNG, JPEG) với các loại khác nhau từ mdpi, hdpi, sw600dp... cho tới resource ngôn ngữ.
- AndroidManifest.xml: Mô tả tên, phiên bản, và nội dung của tập tin apk.
- classes.dex: Bao gồm mã nguồn đã biên dịch, được chuyển dưới dạng Dex bytecode
- resources.arsc: Một số tài nguyên và định nghĩa được biên dịch và lưu trữ ở đây. Các resource này lưu vào apk mà không cần nén để truy cập nhanh hơn lúc runtime.
- Apk là một tập tin nén (zip format-type) được dựa trên JAR file format với đuôi .apk là phần mở rộng. Uzip tập tin apk:

```

phaphajian@PhaPha-Jians-MacBook-Pro: ~/Desktop/Android
unzip InsecureBankv2.apk
Archive:  InsecureBankv2.apk
  inflating: AndroidManifest.xml
  inflating: res/anim/abc_fade_in.xml
  inflating: res/anim/abc_fade_out.xml
  inflating: res/anim/abc_grow_fade_in_from_bottom.xml
  inflating: res/anim/abc_popup_enter.xml
  inflating: res/anim/abc_popup_exit.xml
  inflating: res/anim/abc_shrink_fade_out_from_bottom.xml
  inflating: res/anim/abc_slide_in_bottom.xml
  inflating: res/anim/abc_slide_in_top.xml
  inflating: res/anim/abc_slide_out_bottom.xml
  inflating: res/anim/abc_slide_out_top.xml
  inflating: res/color-v11/abc_background_cache_hint_selector_material_dark.xml
  inflating: res/color-v11/abc_background_cache_hint_selector_material_light.xml
  
```

- Các thành phần nói trên đã xuất hiện

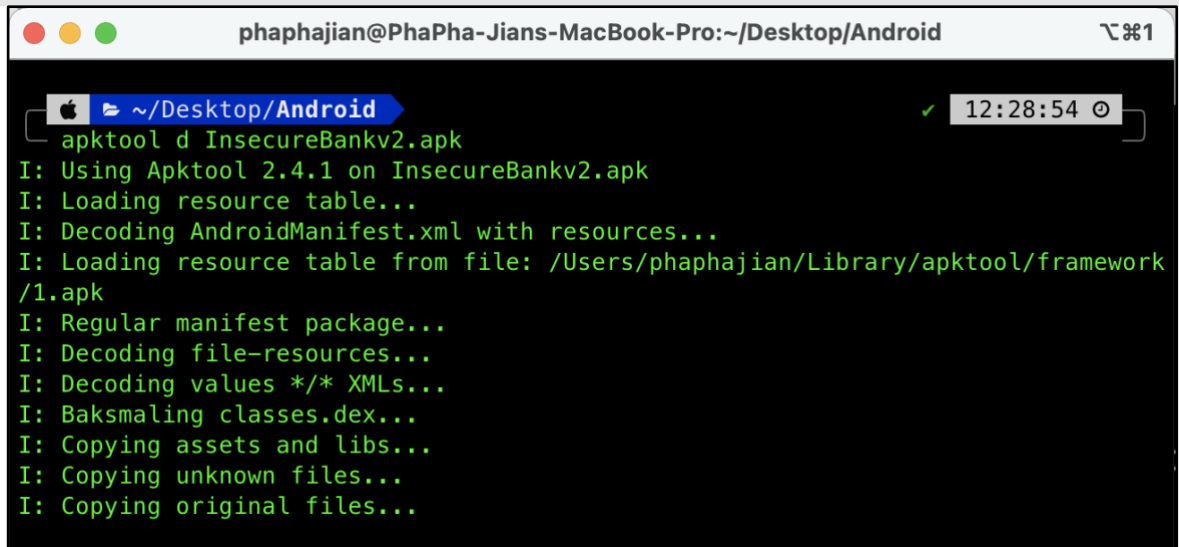
```

phaphajian@PhaPha-Jians-MacBook-Pro: ~/Desktop/Android/InsecureBankv2
ls -al
total 12872
drwxr-xr-x@ 7 phaphajian  staff   224 Apr 24 12:21 .
drwxr-xr-x@ 25 phaphajian  staff   800 Apr 24 12:21 ..
-rw-r--r--@ 1 phaphajian  staff  7588 Sep 18  2015 AndroidManifest.xml
drwxr-xr-x@ 5 phaphajian  staff   160 Apr 24 12:21 META-INF
-rw-r--r--@ 1 phaphajian  staff 6125692 Sep 18  2015 classes.dex
drwxr-xr-x@ 27 phaphajian  staff   864 Apr 24 12:21 res
-rw-r--r--@ 1 phaphajian  staff 454300 Jul 26  2015 resources.arsc
  
```

- ⇒ Phần này chưa gọi là dịch ngược vì nó còn ở dạng binary.
- ⇒ Deomplie: phiên dịch ngược mã nguồn.
- apktool được sử dụng để decompile (Chi tiết cài đặt [tại đây](#))
- apktool giúp decompile tập AndroidManifest.xml về tập tin .xml nguyên bản, cũng như resources.arsc và classes.dex thành một dạng ngôn ngữ gọi là SMALI.

- Thực hiện lệnh sau:

```
apktool d InsecureBankv2.apk
```

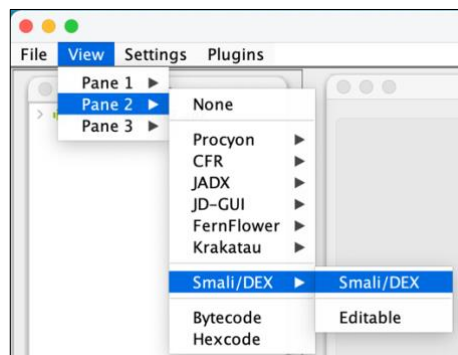


```
phaphajian@PhaPha-Jians-MacBook-Pro:~/Desktop/Android
~ /Desktop/Android
apktool d InsecureBankv2.apk
I: Using Apktool 2.4.1 on InsecureBankv2.apk
I: Loading resource table...
I: Decoding AndroidManifest.xml with resources...
I: Loading resource table from file: /Users/phaphajian/Library/apktool/framework/1.apk
I: Regular manifest package...
I: Decoding file-resources...
I: Decoding values */* XMLs...
I: Baksmaling classes.dex...
I: Copying assets and libs...
I: Copying unknown files...
I: Copying original files...
```

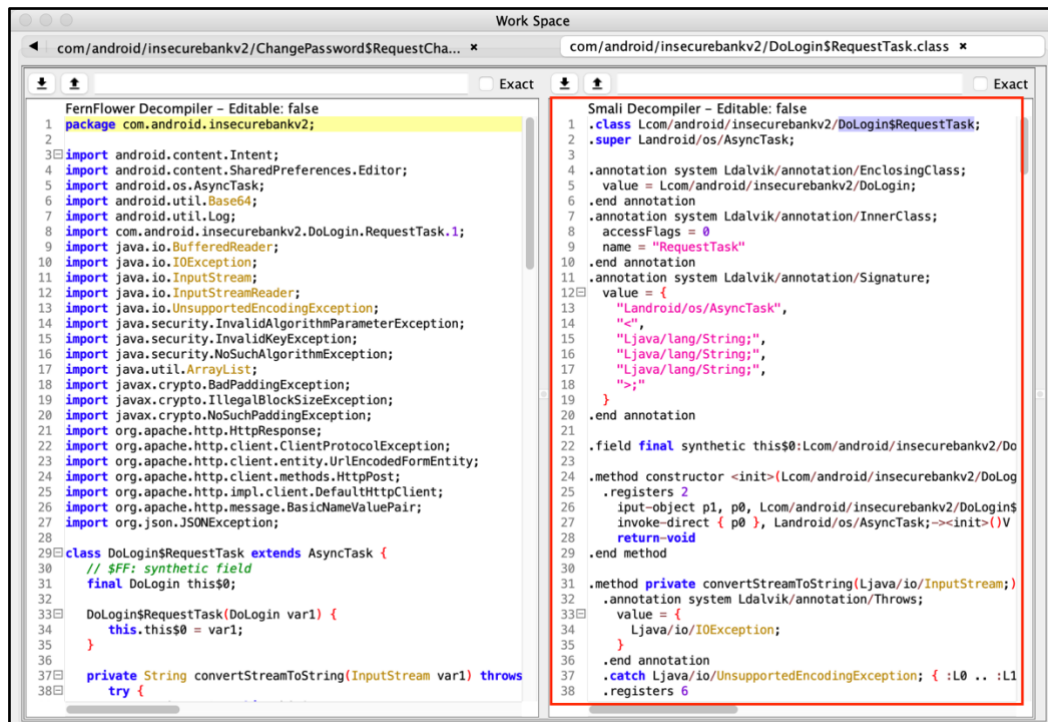
- Bước kế tiếp ta có thể lấy được mã nguồn bằng tool dex2jar. Sau đó đọc nội dung các tập tin jar bằng công cụ jd-gui.
- ⇒ Nhằm tự động hoá các quá trình trên ta dùng Bytecode Viewer.

C.2.2 Smali

Thêm tập tin apk vào Bytecode Viewer.



Mở tập tin *InsecureBankv2/com/android/insecurebanv2/DoLogin\$RequestTask.class*, ta sẽ thấy smali code ở tab/panel thứ 2



Phần trên có đoạn “devadmin”, xem thử smali thì nó hình thù như thế nào.

```

117▢ if (this.this$0.username.equals("devadmin")) {
118    var5.setEntity(new UrlEncodedFormEntity(var4));
119    var6 = var2.execute(var5);
120▢ } else {
121    var3.setEntity(new UrlEncodedFormEntity(var4));
122    var6 = var2.execute(var3);
123 }

```

```

283 const-string v5, "devadmin"
284 invoke-virtual { v4, v5 }, Ljava/lang/String; ->equals(Ljava/lang/Object;)Z
285 move-result v4
286 if-eqz v4, :L2

```

- Nó sẽ lấy chuỗi ta nhập vào (v4) và so sánh chuỗi “devadmin” (v5) bằng cách gọi method `java.lang.String.equals()`, nếu đúng trả về 1 sai trả về 0.
- Sau đó lưu kết quả trên vào v4.
- Tiếp tục chương trình gọi câu lệnh `if-eqz` (nếu bằng 0 – if equals zero), nghĩa là nếu chuỗi nhập vào không thoả mãn sẽ nhảy đến phần code ở :L2.

- Xem qua đoạn code tiếp theo nếu if-eqz thoả mãn và không thoả mãn.
 - o **Là “devadmin”**

```

if-eqz v4, :L2

new-instance v1, Lorg/apache/http/client/entity/UrlEncodedFormEntity;

invoke-direct { v1, v3 }, Lorg/apache/http/client/entity/UrlEncodedFormEntity;-><init>(Ljava/util/List;)V

invoke-virtual { v2, v1 }, Lorg/apache/http/client/methods/HttpPost;->setEntity(Lorg/apache/http/HttpEntity;)V

invoke-interface { v0, v2 }, Lorg/apache/http/client/HttpClient;-
>execute(Lorg/apache/http/client/methods/HttpRequest;)Lorg/apache/http/HttpResponse;

move-result-object v0

:L0

```

Đoạn java tương đương

```

var5.setEntity(new UrlEncodedFormEntity(var4));
var6 = var2.execute(var5)

```

⇒ v2 là gì?

```

const-string v4, "/devlogin"
invoke-virtual { v3, v4 }, Ljava/lang/StringBuilder;->append(Ljava/lang/String;)Ljava/lang/StringBuilder;

move-result-object v3

invoke-virtual { v3 }, Ljava/lang/StringBuilder;->toString()Ljava/lang/String;

move-result-object v3

invoke-direct { v2, v3 }, Lorg/apache/http/client/methods/HttpPost;-><init>(Ljava/lang/String;)V

```

Đoạn code java tương đương

```

HttpPost var5 = new HttpPost(this.this$0.protocol + this.this$0.serverip + ":" + this.this$0.serverport
+ "/devlogin");

```

⇒ Kết luận đoạn này nếu chúng ta nhập “devadmin”, thì nó truy vấn HTTP POST đến <http://ip server/devlogin>.

- o **Không là là “devadmin”**

```

:L2

new-instance v2, Lorg/apache/http/client/entity/UrlEncodedFormEntity;

invoke-direct { v2, v3 }, Lorg/apache/http/client/entity/UrlEncodedFormEntity;-><init>(Ljava/util/List;)V

invoke-virtual { v1, v2 }, Lorg/apache/http/client/methods/HttpPost;->setEntity(Lorg/apache/http/HttpEntity;)V

```

```

invoke-interface { v0, v1 }, Lorg/apache/http/client/HttpClient;-
>execute(Lorg/apache/http/client/methods/HttpRequest;)Lorg/apache/http/HttpResponse;

move-result-object v0

goto/16 :L0

```

C.2.3 Patching ứng dụng Android

- Patch là tác động và thay đổi code của chương trình, làm cho nó hoạt động theo ý muốn.
- apktool có thể patch ứng dụng.
- Vào thư mục res đã decompile trước đó, Insecurebankv2/res/values/strings.xml

```

73 <string name="create_calendar_message">Allow Ad to create a calendar event?</string>
74 <string name="create_calendar_title">Create calendar event</string>
75 <string name="decline">Decline</string>
76 <string name="hello_world">Hello world!</string>
77 ... <string name="is_admin">no</string>
78 <string name="loginscreen_password">Password:</string>
79 <string name="loginscreen_username">Username:</string>
80 <string name="pref_submit">Submit:</string>
81 <string name="server_ip">Server IP:</string>
82 <string name="server_port">Server Port:</string>
83 <string name="status_bar_notification_info_overflow">999+</string>

```

- Sửa value "is_admin" bằng yes và cái kết???

```

73 <string name="create_calendar_message">Allow Ad to create a calendar event?</string>
74 <string name="create_calendar_title">Create calendar event</string>
75 <string name="decline">Decline</string>
76 <string name="hello_world">Hello world!</string>
77 ... <string name="is_admin">yes</string>
78 <string name="loginscreen_password">Password:</string>
79 <string name="loginscreen_username">Username:</string>
80 <string name="pref_submit">Submit:</string>
81 <string name="server_ip">Server IP:</string>
82 <string name="server_port">Server Port:</string>
83 <string name="status_bar_notification_info_overflow">999+</string>

```

- Bây giờ recompile lại tập tin apk

```

~/Desktop/Android
apktool b InsecureBankv2 -o InsecureBankv3.apk
I: Using Apktool 2.4.1
I: Checking whether sources has changed...
I: Smaling smali folder into classes.dex...
I: Checking whether resources has changed...
I: Building resources...
I: Building apk file...
I: Copying unknown files/dir...
I: Built apk...

```

- Sau đó cài đặt vào điện thoại

```

~/Desktop/Android
adb install InsecureBankv3.apk
Performing Streamed Install
adb: failed to install InsecureBankv3.apk: Failure [INSTALL_PARSE_FAILED_NO_CERTIFICATES: Failed to collect certificates from /data/app/vmdl67017203.tmp/base.apk: Attempt to get length of null array]

```

⇒ Failed rồi! Android yêu cầu các tập tin APK đều phải được ký bằng một chứng chỉ trước khi được phép cài đặt trên thiết bị. Sau khi chỉnh sửa, tập tin APK sẽ không còn toàn vẹn như ban đầu nên cần phải được ký lại.

- Tạo Keystore:

```

keytool -genkey -v -keystore InsecureBankv3.keystore -alias InsecureBankv3 -keyalg RSA -keysize 2048 -validity 10000
keytool -genkey -v -keystore InsecureBankv3.keystore -alias InsecureBankv3 -keyalg RSA -keysize 2048 -validity 10000

Enter keystore password:
Re-enter new password:
What is your first and last name?
[Unknown]:
What is the name of your organizational unit?
[Unknown]:
What is the name of your organization?
[Unknown]:
What is the name of your City or Locality?
[Unknown]:
What is the name of your State or Province?
[Unknown]:
What is the two-letter country code for this unit?
[Unknown]:
Is CN=Unknown, OU=Unknown, O=Unknown, L=Unknown, ST=Unknown, C=Unknown correct?
[no]: yes

Generating 2,048 bit RSA key pair and self-signed certificate (SHA256withRSA) with a validity of 10,000 days
for: CN=Unknown, OU=Unknown, O=Unknown, L=Unknown, ST=Unknown, C=Unknown
[Storing InsecureBankv3.keystore]

```

- Sử dụng công cụ **apksigner** hoặc **jarsigner**:

```

apksigner sign --ks InsecureBankv3.keystore InsecureBankv3.apk

~/Desktop/Android
/Users/phaphajian/Library/Android/sdk/build-tools/30.0.2/apksigner sign --ks InsecureBankv3.keystore InsecureBankv3.apk
Keystore password for signer #1:

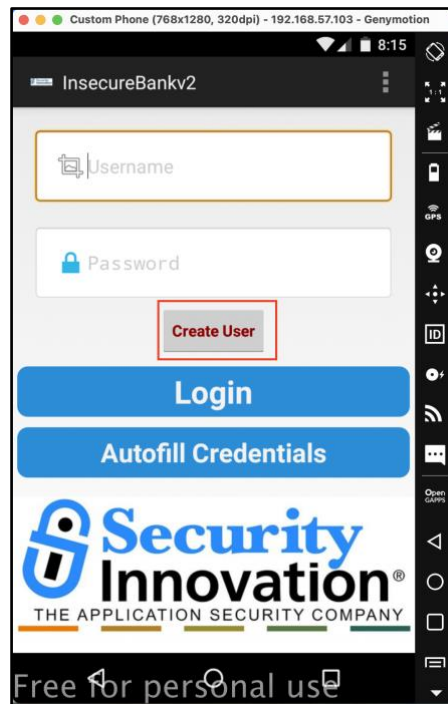
```

- Tiến hành cài lại và mở ứng dụng

```

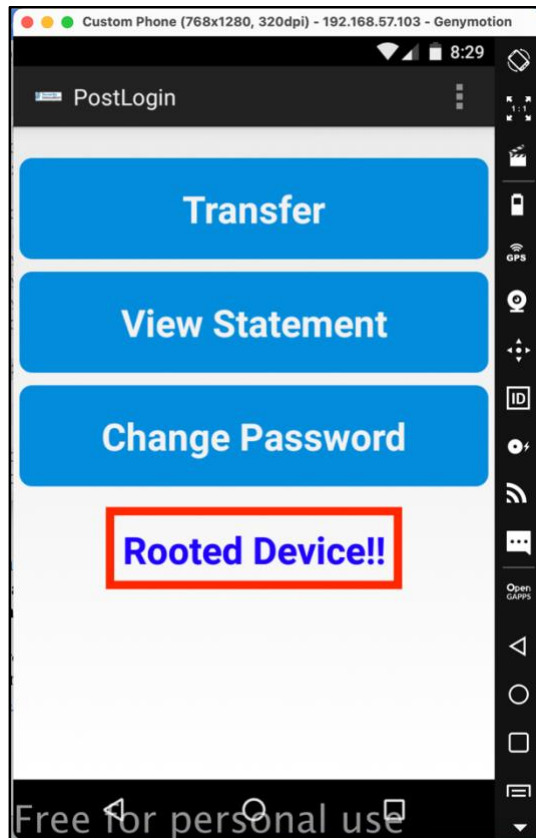
~/Desktop/Android
adb install InsecureBankv3.apk
Performing Streamed Install
Success

```

⇒ Nút Create User xuất hiện chứng tỏ chỉ có admin mới có.

- Các chương trình đều có chức năng chặn phân tích như:
 - Root Detection (anti-root)
 - Emulator Detection (anti-vm)



- Đoạn check root nằm trong class PostLogin.class

```
80 void showRootStatus() {  
81     boolean var1;  
82     if (!this.doesSuperuserApkExist("/system/app/Superuser.apk") && !this.doesSUexist()) {  
83         var1 = false;  
84     } else {  
85         var1 = true;  
86     }  
87  
88     if (var1) {  
89         this.root_status.setText("Rooted Device!!");  
90     } else {  
91         this.root_status.setText("Device not Rooted!!");  
92     }  
93 }  
94 }
```

- Đoạn mã kiểm tra sự tồn tại của "/system/app/Superuser.apk"???
- Để kiểm chứng ta vào shell của điện thoại và tìm kiếm

```
vbox86p:/ # ls -la /system/app/Superuser/Superuser.apk  
/system/app/Superuser/Superuser.apk
```

⇒ Kiểm tra sai đường dẫn ☺)))))))))

Yêu cầu 6 Sinh viên điều chỉnh mã nguồn ứng dụng sao cho luôn hiển thị trạng thái “Rooted Device!!” với bất kỳ trạng thái nào của thiết bị.

Gợi ý:

```

422 .method showRootStatus()V
423 .locals 3
424
425 .prologue
426 const/4 v1, 0x1
427
428 .line 86
429 const-string v2, "/system/app/Superuser.apk"
430
431 invoke-direct {p0, v2}, Lcom/android/insecurebankv2/PostLogin;->doesSuperuserApkExist(Ljava/lang/String;)Z
432
433 move-result v2
434
435 if-nez v2, :cond_0
436
437 .line 87
438 invoke-direct {p0}, Lcom/android/insecurebankv2/PostLogin;->doesSUexist()Z
439
440 move-result v2
441
442 if-eqz v2, :cond_1
443
444 :cond_0
445 move v0, v1
446
447 .line 88
448 .local v0, "isrooted":Z
449 :goto_0
450 if-ne v0, v1, :cond_2
451
452 .line 90
453 iget-object v1, p0, Lcom/android/insecurebankv2/PostLogin;->root_status:Landroid/widget/TextView;
454
455 const-string v2, "Rooted Device!!"
456
457 invoke-virtual {v1, v2}, Landroid/widget/TextView;->setText(Ljava/lang/CharSequence;)V
458
459 .line 96
460 :goto_1
461 return-void
462
463 .line 87
464 .end local v0 # "isrooted":Z
465 :cond_1
466 const/4 v0, 0x0
467
468 goto :goto_0
469
470 .line 94
471 .restart local v0 # "isrooted":Z
472 :cond_2
473 iget-object v1, p0, Lcom/android/insecurebankv2/PostLogin;->root_status:Landroid/widget/TextView;
474
475 const-string v2, "Device not Rooted!!"
476
477 invoke-virtual {v1, v2}, Landroid/widget/TextView;->setText(Ljava/lang/CharSequence;)V
478
479 goto :goto_1
480 .end method

```

if condition

C.3 Phân tích động (Dynamic Analysis)

- Phân tích động là quá trình kiểm thử và đánh giá trong thời gian thực (real time).
- Mục đích của phân tích tĩnh theo là theo dõi biểu hiện trao đổi dữ liệu giữa chương trình và internet mà thay vì ngồi đọc mã nguồn.
- Công cụ sẽ hỗ trợ tiến hành phân tích là **Burp Suite**. Burp Suite là một ứng dụng java chạy đa nền tảng (Windows/Linux/Mac) dùng để kiểm thử xâm nhập ứng dụng Web.
- Cài đặt tại: <https://portswigger.net/burp/communitydownload>



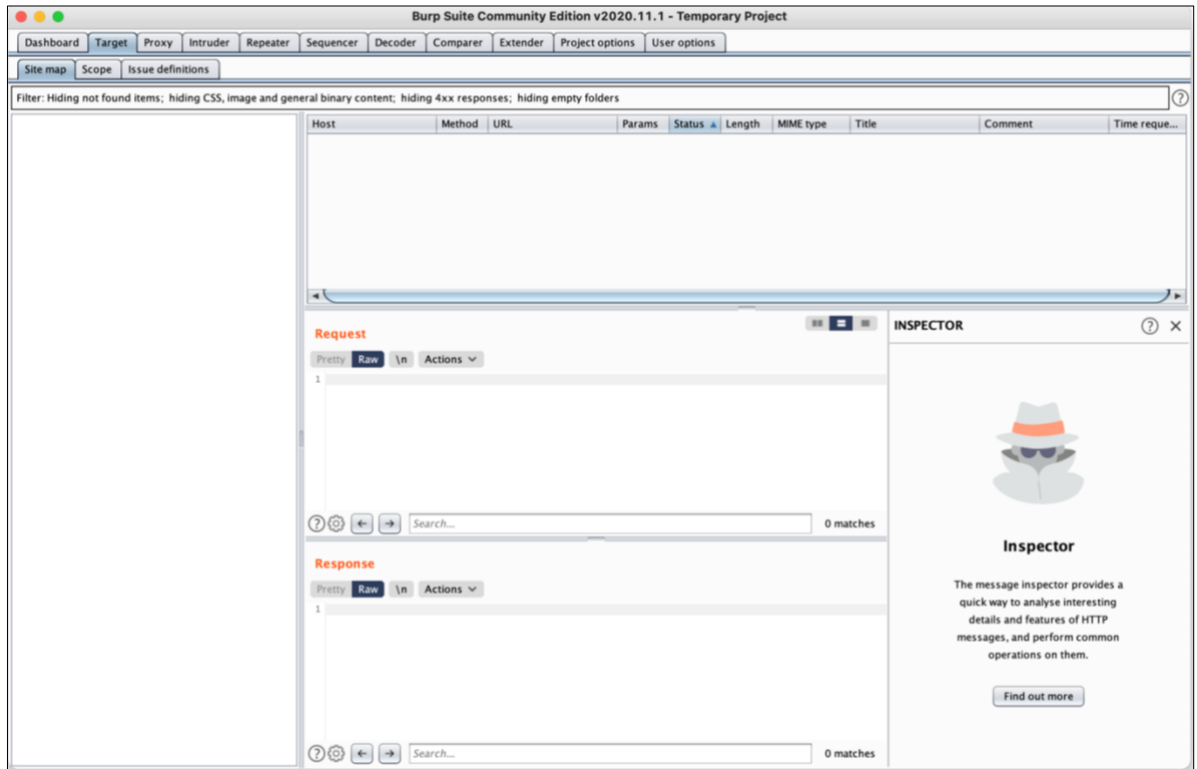
C.3.1 Cài đặt

Cấu hình cho điện thoại ảo “proxy” thông qua Burp Suite, điều này có nghĩa là mọi yêu cầu (request) được gửi từ điện thoại đều được Burp Suite bắt lại, rồi mới đi đến nơi cần đến (server).

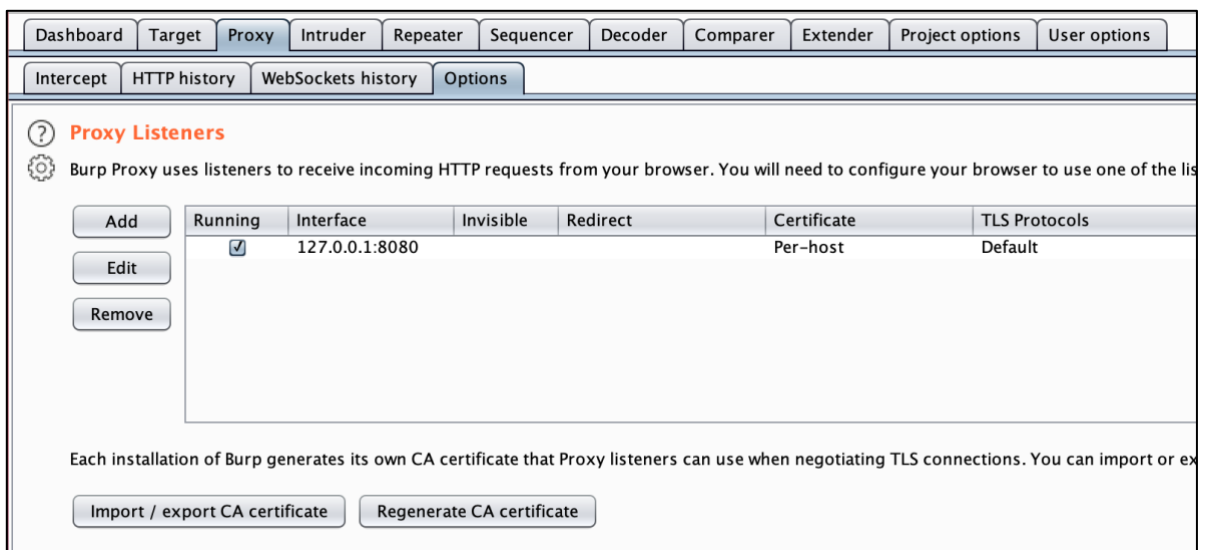
1. Kiểm tra địa chỉ ip của điện thoại ảo

```
> adb devices
List of devices attached
192.168.56.103:5555    device
```

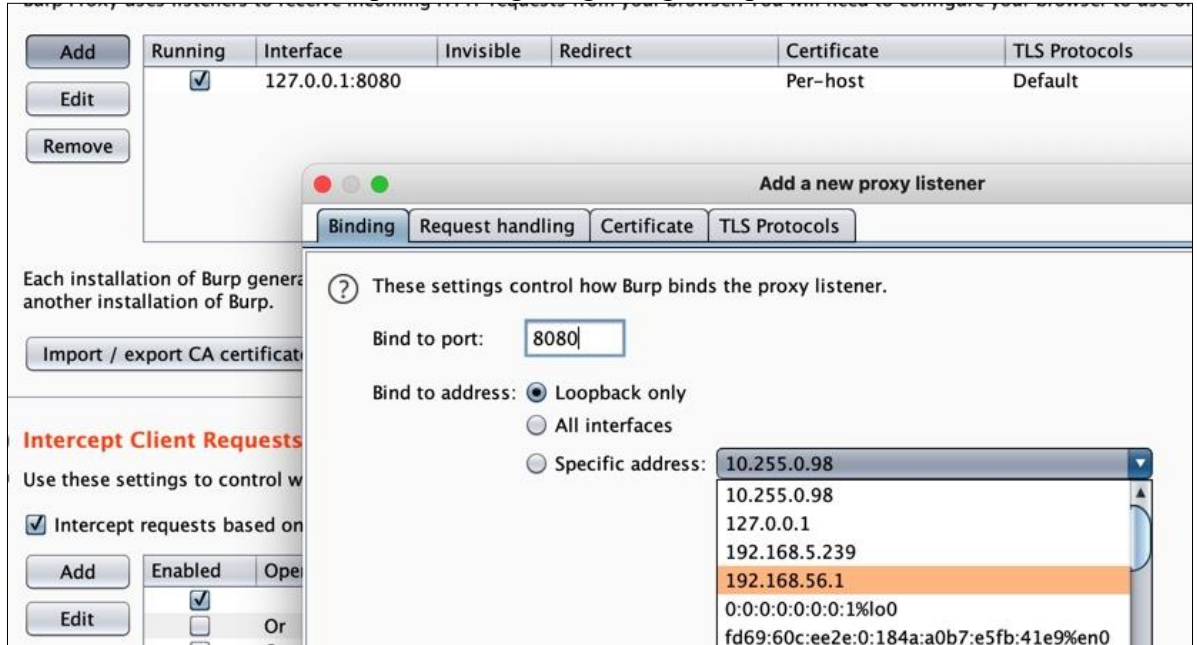
2. Mở Burp Suite



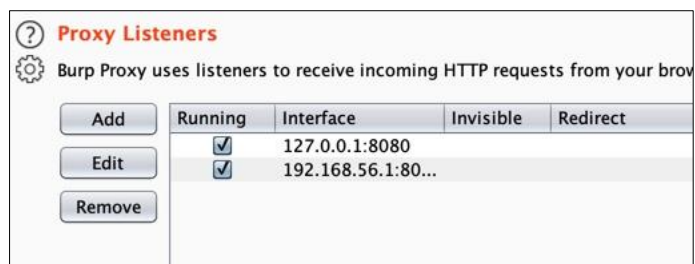
3. Chọn Tab *Proxy* > *Options*



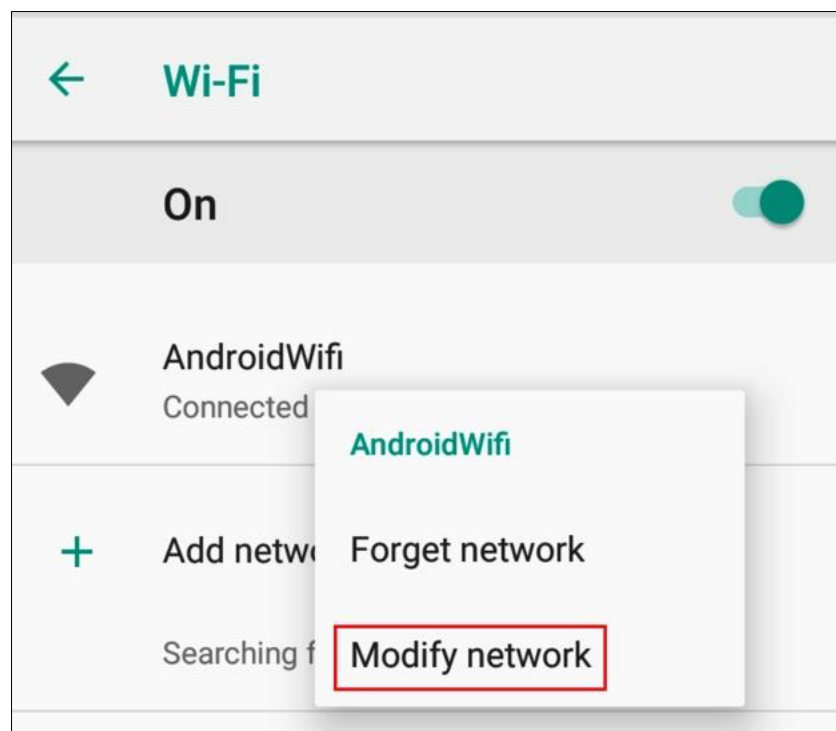
4. Chọn *Add*, chọn địa chỉ ip nằm trong vùng mạng của ip điện thoại ảo



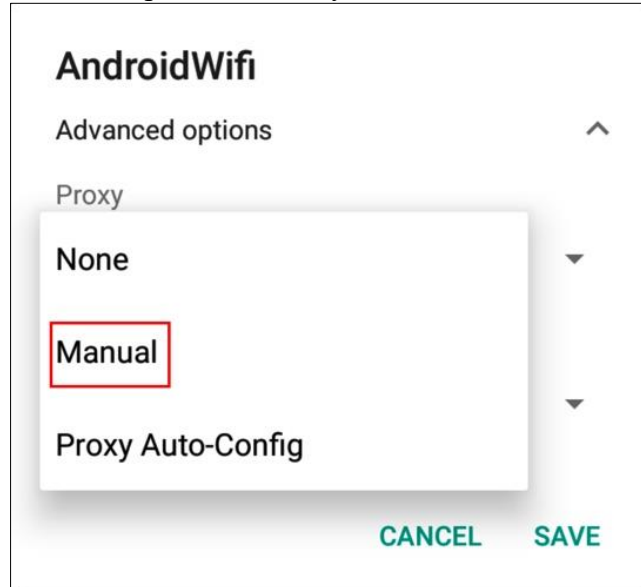
5. *Proxy Listeners* đã xuất hiện interface vừa tạo, chọn tick phần *Running*.



6. Ở điện thoại ảo vào *Settings > Network & Internet > Wifi*, click giữ vào trường tên *SSID*. Chọn *Modify network*

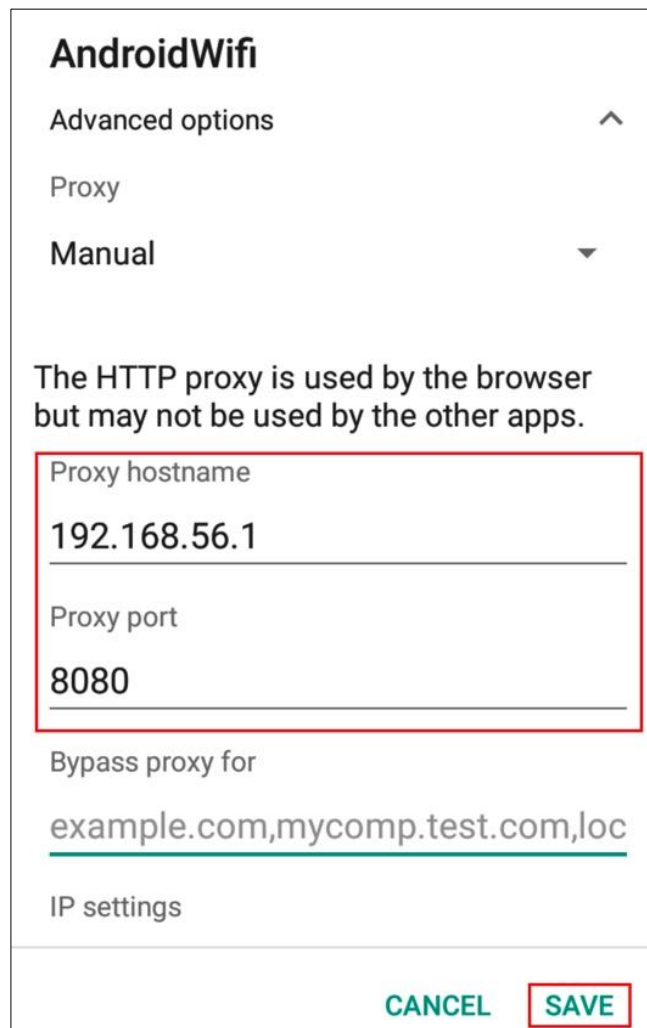


7. Chọn xổ xuống Advanced options. Ở Proxy chọn Manual



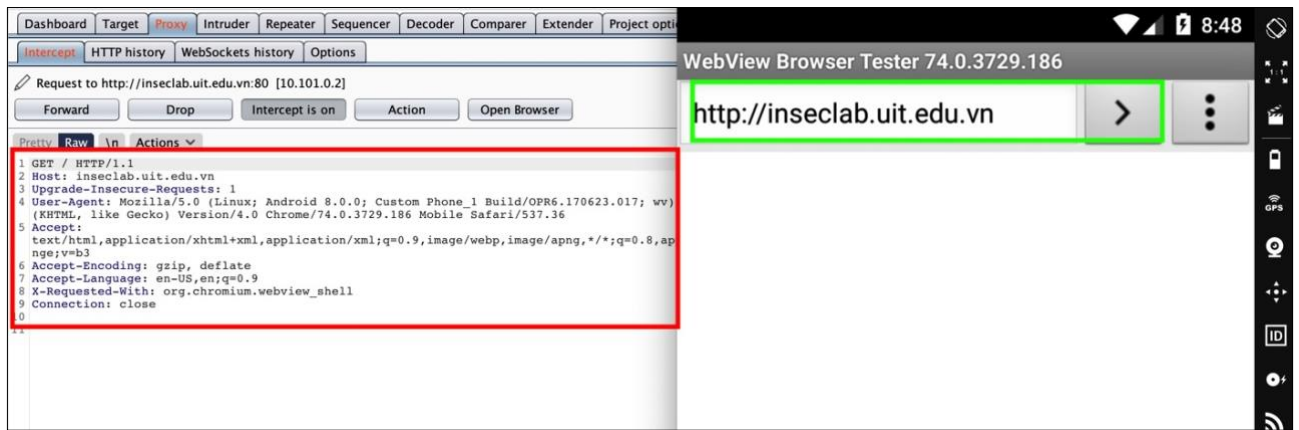
The screenshot shows the 'AndroidWifi' settings window. Under 'Advanced options', the 'Proxy' dropdown menu is open, displaying three options: 'None', 'Manual' (which is highlighted with a red box), and 'Proxy Auto-Config'. At the bottom right of the window are 'CANCEL' and 'SAVE' buttons.

8. Điền thông tin proxy đã thiết lập bên trên



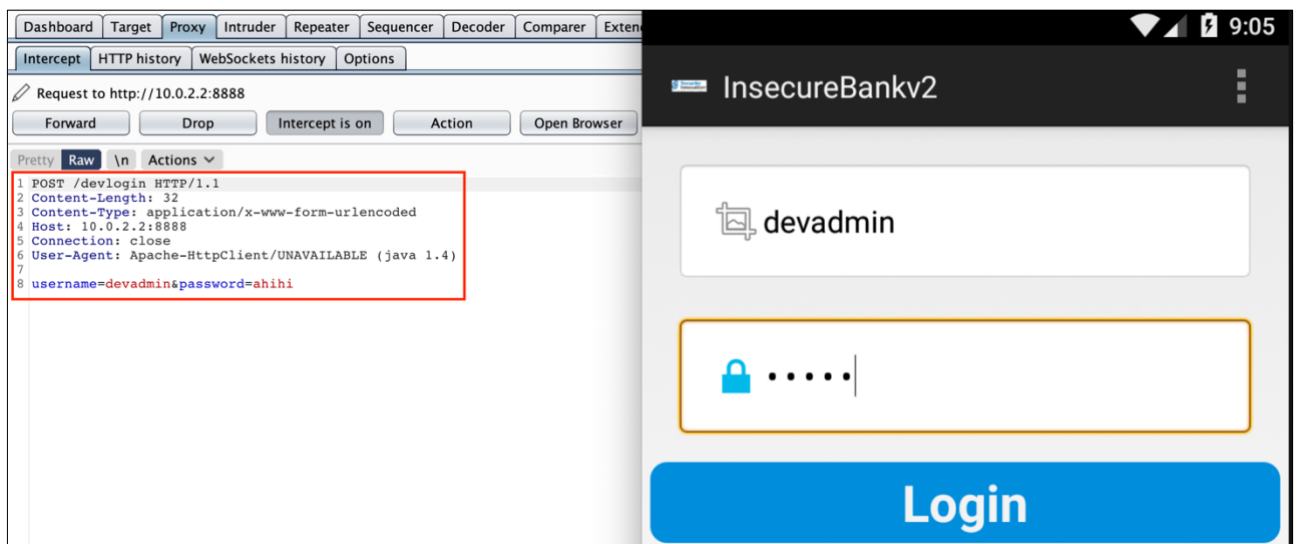
The screenshot shows the 'AndroidWifi' settings window with 'Manual' selected under 'Proxy'. Below this, a text box explains: 'The HTTP proxy is used by the browser but may not be used by the other apps.' The 'Proxy hostname' field is filled with '192.168.56.1' and the 'Proxy port' field is filled with '8080', both fields are highlighted with a red box. Below these, the 'Bypass proxy for' field contains 'example.com,mycomp.test.com,loc'. At the bottom right are 'CANCEL' and 'SAVE' buttons, with 'SAVE' highlighted by a red box.

9. Hoàn tất cấu hình, truy cập thử một trang web

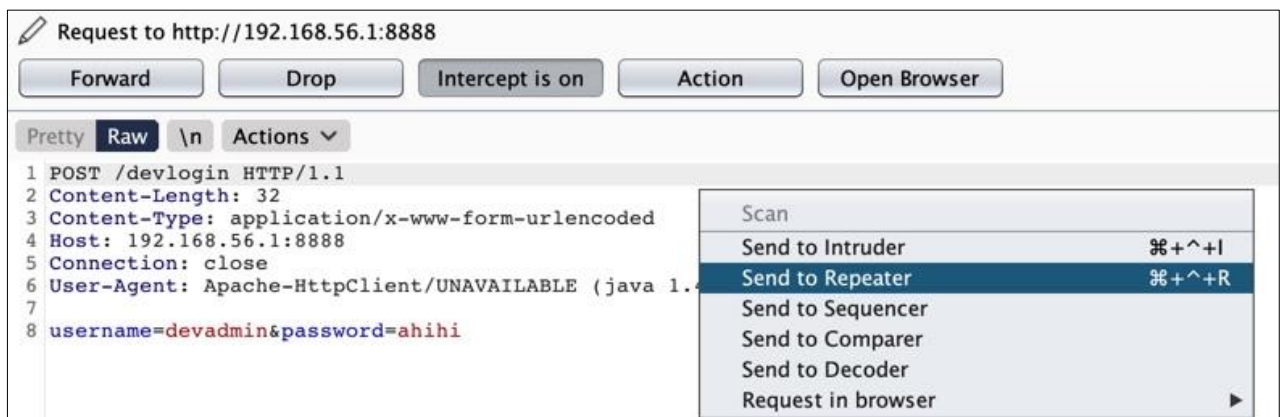


C.3.2 Ứng dụng

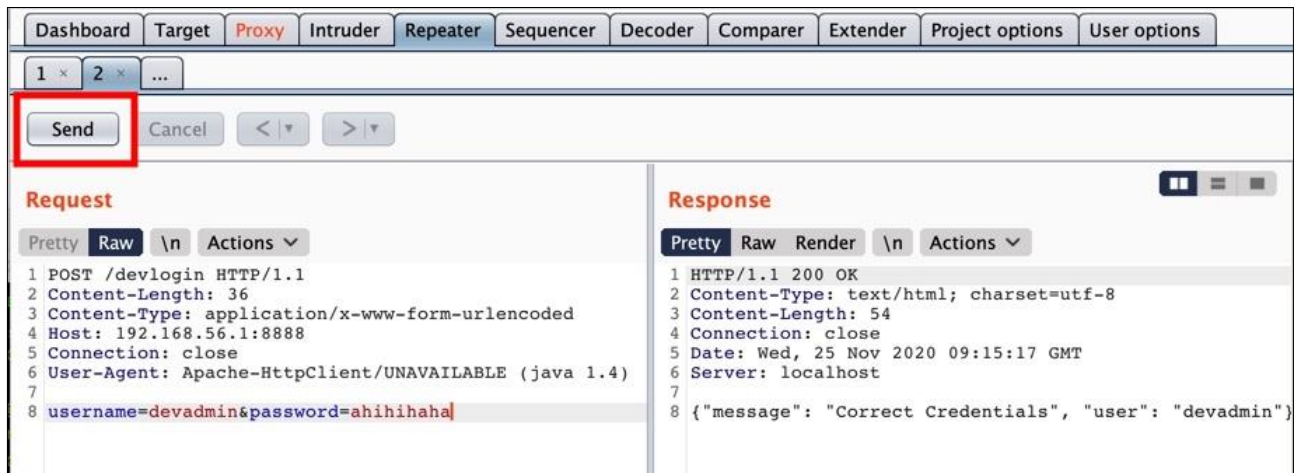
Sử dụng ứng dụng Insecurebankv2, khi đó tiến trình bị bắt lại, ta có thể thay đổi các trường của payload



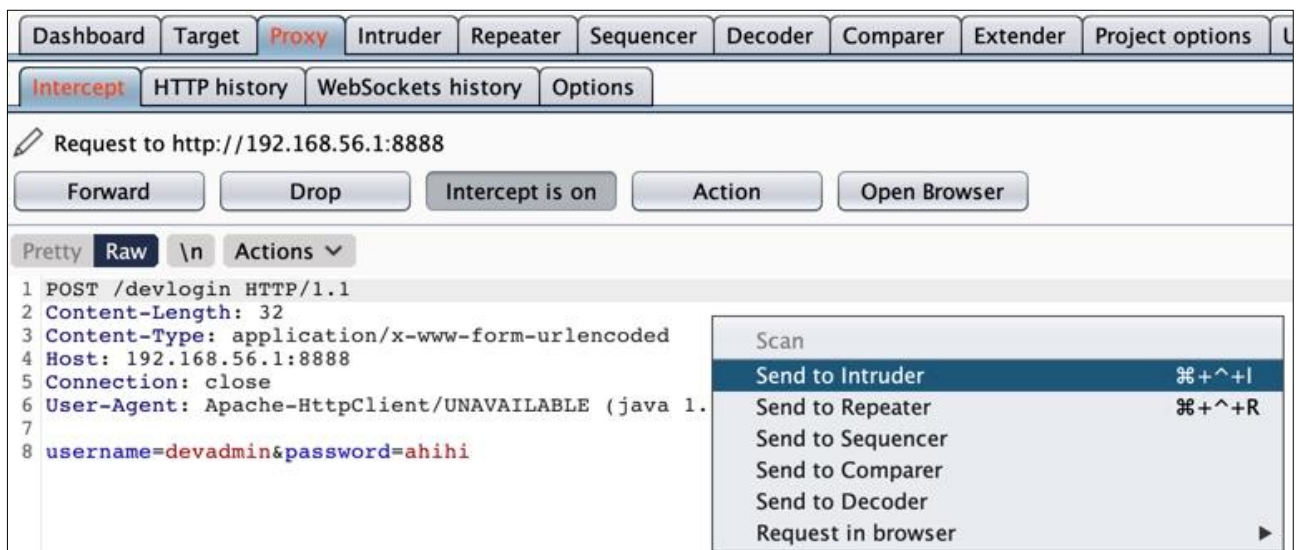
Sử dụng chức năng Repeater, tính năng giúp tiết kiệm thời gian cho việc phải bắt lại request nhiều lần



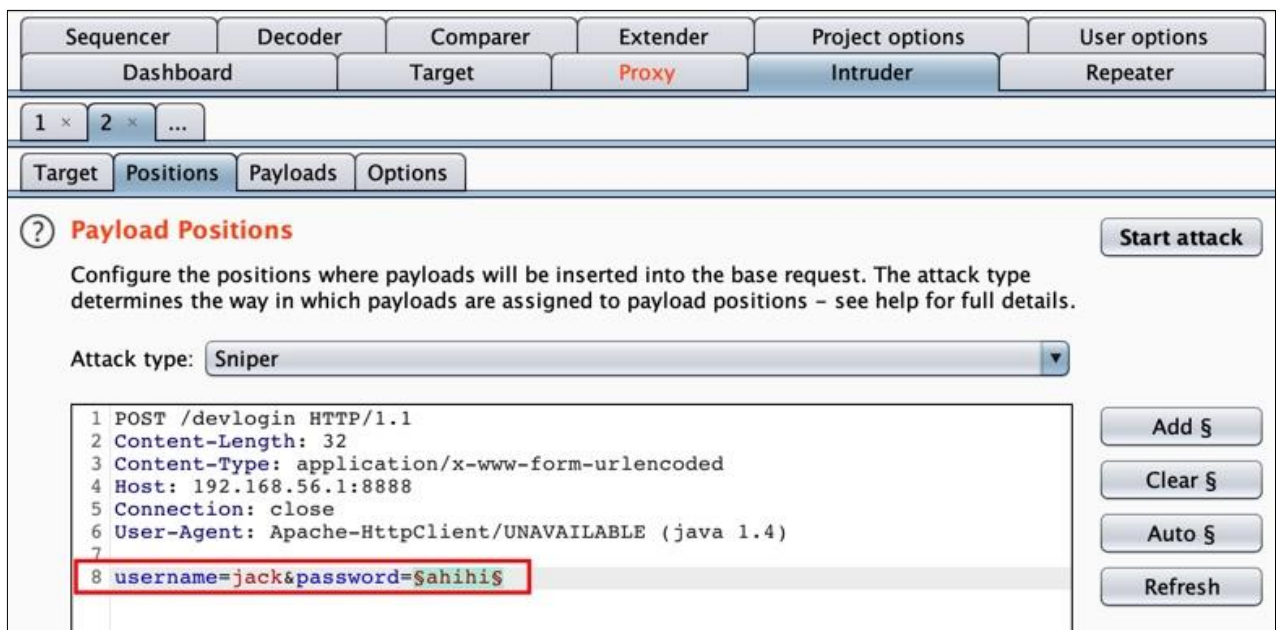
Chọn *Send* vào Repeater, và sử dụng nhiều lần như thế ở tab này



Tấn công vét cạn mật khẩu bằng cách gửi request vào tab *Intruder*



Vét cạn mật khẩu cho tài khoản “jack”. Ký hiệu \$ để định dạng cho phần vét cạn



Tại tab *Intruder* > *Payloads*, ở *Payload Option* là ta thêm phần payload để vét cạn vào vị trí password.

Target **Positions** **Payloads** **Options**

Payload Sets

You can define one or more payload sets. The number of payload sets depends on the attack type defined in the Positions tab. Various payload types are available for each payload set, and each payload type can be customized in different ways.

Payload set: 1 Payload count: 8

Payload type: Simple list Request count: 8

Start attack

Payload Options [Simple list]

This payload type lets you configure a simple list of strings that are used as payloads.

Paste Load ... Remove Clear Add

123
abc
Dinesh@123\$
ahihi
jack@123
Jack@123\$
dinesh
aaaaaaa

Add from list ... [Pro version only]

Có một request có độ khác với request còn lại

Results Target Positions Payloads Options

Filter: Showing all items

Request	Payload	Status	Error	Timeout	Length	Comment
0		200			199	
1	123	200			199	
2	abc	200			199	
3	Dinesh@123\$	200			199	
4	jack	200			199	
5	Jack@123\$	200			204	
6	aaaaaaa	200			199	
7	dinesh	200			199	

Text

Request Response

Pretty Raw Render \n Actions

```

1 HTTP/1.1 200 OK
2 Content-Type: text/html; charset=utf-8
3 Content-Length: 50
4 Connection: close
5 Date: Wed, 25 Nov 2020 09:40:36 GMT
6 Server: localhost
7
8 {"message": "Correct Credentials", "user": "jack"}

```

Phần *History* hiển thị lịch sử của các request được gửi từ điện thoại ảo

Intercept HTTP history WebSockets history Options

Filter: Hiding CSS, image and general binary content

#	Host	Method	URL	Params	Edited	Status	Length	MIME type
1	http://tsu0d.com	GET	/					
3	http://inseclab.uit.edu.vn	GET	/					
4	http://inseclab.uit.edu.vn	GET	/					
5	http://inseclab.uit.edu.vn	GET	/favicon.ico			301	242	HTML
6	http://10.0.2.2:8888	POST	/devlogin		✓			
7	http://192.168.56.1:8888	POST	/devlogin		✓	200	208	JSON
8	http://192.168.56.1:8888	POST	/devlogin		✓	200	208	JSON
9	http://192.168.56.1:8888	POST	/devlogin		✓	200	208	JSON
10	http://192.168.56.1:8888	POST	/devlogin		✓	200	208	JSON
11	http://192.168.56.1:8888	POST	/devlogin		✓	200	208	JSON
12	http://192.168.5.81:8888	POST	/login		✓			
13	http://192.168.5.81:8888	POST	/login		✓			
14	http://192.168.56.1:8888	POST	/login		✓	200	199	JSON

C.3.3 Kiểm tra rò rỉ dữ liệu qua Logcat

Yêu cầu 7 Hãy thực hiện thao tác chuyển tiền (Transfer) trên app InsecureBankv2, đồng thời quan sát adb logcat. Tìm và chụp lại đoạn log chứa thông tin nhạy cảm bị in ra màn hình console

C.4 Hooking với Frida

Frida là bộ công cụ tự động dành cho lập trình viên, kỹ sư dịch ngược và nghiên cứu viên bảo mật với chức năng: tracing, profiling và debugging.

C.4.1 Cài đặt

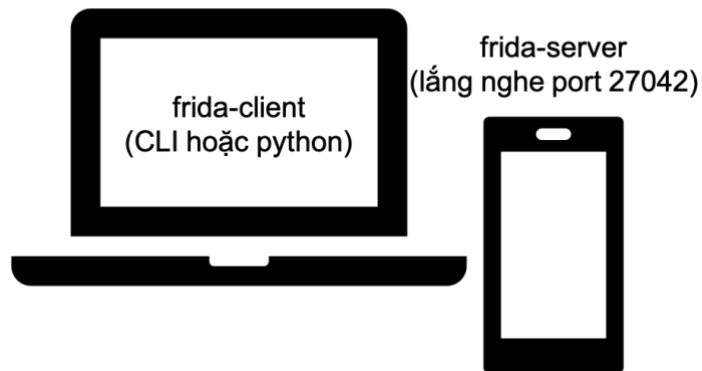
Máy thật dùng python3

```
pip3 install frida-tools  
pip3 install frida
```

```
Apple > /usr/local/bin  
frida  
Usage: frida [options] target  
  
frida: error: target file, process name or pid must be specified  
  
Apple > /usr/local/bin  
python3  
Python 3.8.2 (default, Oct 2 2020, 10:45:42)  
[Clang 12.0.0 (clang-1200.0.32.27)] on darwin  
Type "help", "copyright", "credits" or "license" for more information.  
>>> import frida  
>>> print(frida.__version__)  
14.0.8  
>>>
```

Điện thoại ảo tải frida-server (<https://github.com/frida/frida/releases>), cụ thể frida-server-14.0.8-android-x86.xz

```
Apple > ~/Downloads  
adb push frida-server-14.0.8-android-x86 /data/local/tmp/frida-server  
frida-server-14.0.8-android-x86: 1 file pushed, 0 skipped. 92.1 MB/s (14581988 bytes in 0.151s)  
  
Apple > ~/Downloads  
adb shell  
vbox86p:/ # cd /data/local/tmp/  
vbox86p:/data/local/tmp # chmod +x frida-server  
vbox86p:/data/local/tmp # ./frida-server &  
[1] 1992  
vbox86p:/data/local/tmp #
```



Kiểm tra cài đặt thành công hay chưa, bằng cách gõ lệnh sau:

```
frida-ps -U
PID  Name
-----
1656  adbd
1612  android.ext.services
462   android.hardware.camera.provider@2.4-service
463   android.hardware.configstore@1.0-service
220   android.hardware.gnss@1.0-service
464   android.hardware.graphics.allocation@2.0-service
173   android.hardware.keymaster@3.0-service
465   android.hardware.sensors@1.0-service
466   android.hardware.wifi@1.0-service
461   android.hidl.allocation@1.0-service
1320  android.process.acore
208   audioserver
```

C.4.2 Thực hành cơ bản

Import thư viện Frida:

```
import frida
```

Dùng hàm `get_usb_device()` để lấy thông tin thiết bị:

```
device = frida.get_usb_device()
```

Spawn chương trình lên

```
pid=device.spawn("com.android.insecurebankv2")
device.resume(pid)
```

Bây giờ có được tiến trình của chương trình vừa tạo, thêm pid vào session

```
session=device.attach(pid)
```

Inject đoạn hook_script vào chương trình

```
script=session.creatr_script(hook_script)
script.load()
```

Đoạn hoàn thiện sẽ như thế này:

```
import frida
import time

device = frida.get_usb_device()
pid = device.spawn("com.android.insecurebankv2")
device.resume(pid)

time.sleep(1) # sleep 1 to avoid crash (sometime)

session=device.attach(pid)

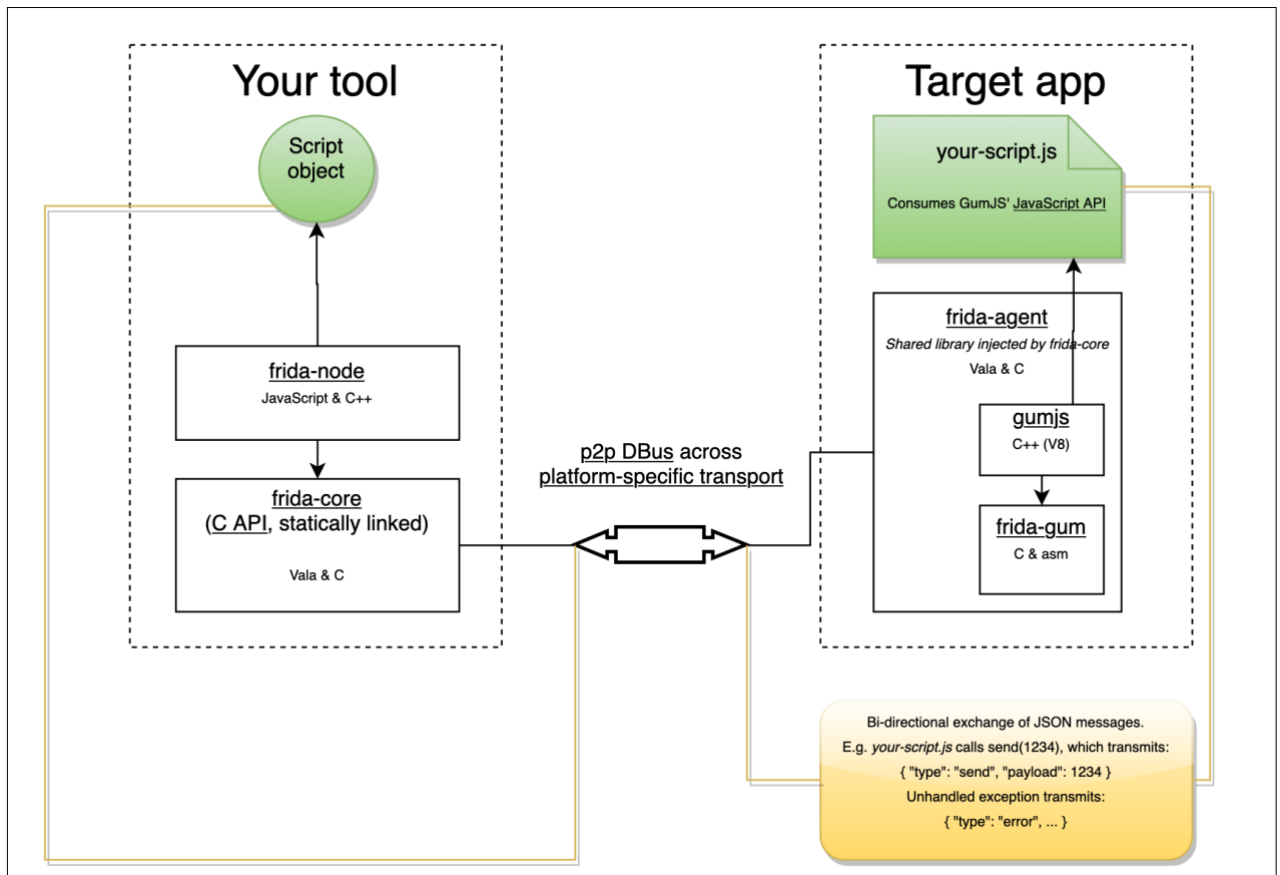
hook_script="""
"""

script=session.create_script(hook_script)
script.load()

input('...?') # prevent terminate
```

hook_script là gì?

Đây sẽ là một tập lệnh cung cấp cho Frida sử dụng Javascript API, điều này giúp tương tác với Java functions và objects.



Đoạn code cung cấp cho Frida sẽ nằm trong **Java.perform(function () {...})**, khi đó hook_script sẽ như sau:

```
hook_script=""
Java.perform(function () {
    # do something
});
""
```

Ở phần *Phân tích tình Yêu cầu 5*, chúng ta phải viết code để giải mã thông điệp, nếu trường hợp không tìm thấy key, hay không rõ thuật toán mã hoá thì sao?

Đoạn code Decrypt-Giải mã tại com/android/inscurebankv2/CryptoClas.class

```
public String aesDecryptedString(String var1) throws UnsupportedOperationException, InvalidKeyException,
NoSuchAlgorithmException, NoSuchPaddingException, InvalidAlgorithmParameterException, IllegalBlockSizeException,
BadPaddingException {
    byte[] var2 = this.key.getBytes("UTF-8");
    this.cipherData = aes256decrypt(this.ivBytes, var2, Base64.decode(var1.getBytes("UTF-8"), 0));
    this.plainText = new String(this.cipherData, "UTF-8");
    return this.plainText;
}
```

- ⇒ Đoạn code nhận vào một đoạn mã hoá crypt-text và trả về đoạn plain-text, suy ra chúng ta không cần quan tâm nó làm gì mà chỉ cần gọi hàm và sử dụng.
- ⇒ Làm thế nào để gọi hàm-function trong Java

Sử dụng **Java.choose()** như sau:

- **Java.choose(className, callbacks)**: enumerate live instances of the **className** class by scanning the Java heap, where **callbacks** is an object specifying:
 - **onMatch(instance)**: called with each live instance found with a ready-to-use **instance** just as if you would have called **Java.cast()** with a raw handle to this particular instance. This function may return the string **stop** to cancel the enumeration early.
 - **onComplete()**: called when all instances have been enumerated

- ⇒ Class sẽ được scan trên Java heap, nếu tìm thấy onMatch() sẽ được thực thi dựa vào đó gọi instance tìm được.

Code code code

```

1  import frida
2  import time
3
4  device = frida.get_usb_device()
5  pid = device.spawn("com.android.insecurebankv2")
6  device.resume(pid)
7
8  time.sleep(1) # sleep 1 to avoid crash (sometime)
9
10 session=device.attach(pid)
11
12 hook_script="""
13 Java.perform
14 (
15     function()
16     {
17         console.log("Inside the hook_script");
18         cryptoClass = Java.choose('com.android.insecurebankv2.CryptoClass',
19         {
20             onMatch : function(instance)
21             {
22                 console.log("Found instance " + instance);
23                 console.log("Result decrypt: " + instance.aesDecryptedString("v/sJpihDCo2ckDmLW5Uwiw=="));
24             },
25             onComplete: function()
26             {
27                 console.log("end");
28             }
29         });
30     }
31 );
32 """
33
34 script=session.create_script(hook_script)
35 script.load()
36
37 input('...?')

```

Kết quả:

```

python3 hook.py
Inside the hook_script
Found instance com.android.insecurebankv2.CryptoClass@214be23
Result decrypt: Jack@123$
end
...?

```

Ta để ý ứng dụng có tính năng phát hiện root, bây giờ sẽ hook và sửa nó. Đoạn code tại `com.andoird/insecurebankv2/PostLogin.class`

```
void showRootStatus() {
    boolean var1;
    if (!this.doesSuperuserApkExist("/system/app/Superuser.apk") && !this.doesSUexist()) {
        var1 = false;
    } else {
        var1 = true;
    }

    if (var1) {
        this.root_status.setText("Rooted Device!!");
    } else {
        this.root_status.setText("Device not Rooted!!");
    }
}
```

`doesSuperuserApkExist()` trả về true thì root detected. Hàm này trả về giá trị boolean, ở đây ta hijack giá trị trả về ???

```
private boolean doesSUexist() {
    // $FF: Couldn't be decompiled
}

private boolean doesSuperuserApkExist(String var1) {
    boolean var2 = true;
    if (!Boolean.valueOf((new File("/system/app/Superuser.apk")).exists())) {
        var2 = false;
    }

    return var2;
}
```

Sử dụng `Java.use()`, cho phép chúng ta ghi đè một phương-thức-method trong class.

- `Java.use(className)` : dynamically get a JavaScript wrapper for `className` that you can instantiate objects from by calling `$new()` on it to invoke a constructor. Call `$dispose()` on an instance to clean it up explicitly (or wait for the JavaScript object to get garbage-collected, or script to get unloaded). Static and non-static methods are available, and you can even replace a method implementation and throw an exception from it:

```
Java.perform(() => {  
  const Activity = Java.use('android.app.Activity');  
  const Exception = Java.use('java.lang.Exception');  
  Activity.onResume.implementation = function () {  
    throw Exception.$new('Oh noes!');  
  };  
});
```

Đầu tiên, chọn class có method đó

```
classPostLogin = Java.use('com.android.insecurebankv2.PostLogin')
```

Sử dụng implementation để ghi đè

```
classPostLogin.doesSuperuserApkExist.implementation = function()  
{  
  // làm cái gì đó trong trường hợp này, return true  
}
```

Code code code

```

1  import frida
2  import time
3
4  device = frida.get_usb_device()
5  pid = device.spawn("com.android.insecurebankv2")
6  device.resume(pid)
7
8  time.sleep(1) # sleep 1 to avoid crash (sometime)
9
10 session=device.attach(pid)
11
12 hook_script="""
13 Java.perform
14 (
15     function()
16     {
17         console.log("Inside the hook_script");
18         classPostLogin = Java.use('com.android.insecurebankv2.PostLogin');
19         classPostLogin.doesSuperuserApkExist.implementation = function()
20         {
21             // làm cái gì đó trong trường hợp này, return true
22         };
23     }
24 );
25 """
26
27 script=session.create_script(hook_script)
28 script.load()
29
30 input('...?')

```

Yêu cầu 8 Hoàn thiện đoạn code trên và demo.

C.4.3 Bypass SSL Pinning

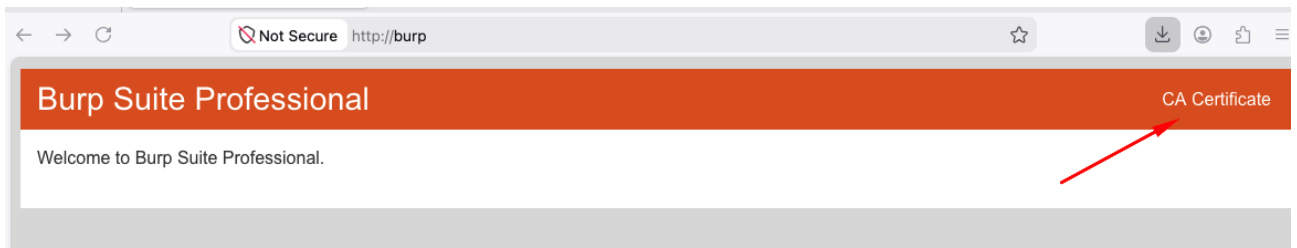
Certificate Pinning là cơ chế bảo mật mà ứng dụng Android từ chối sự tin cậy mặc định của hệ điều hành đối với các Certificate Authority (CA) và chỉ chấp nhận giao tiếp với máy chủ nếu chứng chỉ của máy chủ khớp với chứng chỉ hoặc mã băm (hash) đã được "ghim" cứng (hardcode) trong ứng dụng.

Khi thực hiện phân tích động (Dynamic Analysis) bằng Burp Suite, nếu ứng dụng có cài đặt Certificate Pinning, ứng dụng sẽ từ chối chứng chỉ giả mạo của Burp Suite, dẫn đến kết nối bị ngắt và bạn không thể nhìn thấy các gói tin HTTP/HTTPS. Do đó, để bắt được lưu lượng mạng, chúng ta bắt buộc phải vô hiệu hóa (bypass) cơ chế này.

Đối với thử nghiệm Black-box, có hai hướng tiếp cận chính: Phân tích động (Dynamic) và Phân tích tĩnh (Static).

Để có thể bypass SSL pinning, đầu tiên cần hiểu rõ ứng dụng hiện tại đang pinning như thế nào, ở đây chúng ta sử dụng file [pinning-demo.apk](#)

Cấu hình Burp + cài CA cert vào Android



Sau khi tải file cert về đổi tên thành **cacert.cer**

Cài CA vào Android:

Cách đơn giản (user cert) — dễ nhưng chỉ hoạt động với app có `network_security_config` cho phép user cert hoặc `targetSdk < 24`:

1. Push file `cacert.cer` lên thiết bị: `adb push cacert.cer /sdcard/`
2. Settings → Security → Install from storage → chọn file.

Đối với phiên bản sdk hiện tại đang demo (33) sẽ thực hiện bằng cách 2 (cần root)

→ BMW `adb shell getprop ro.build.version.sdk`

33

→ BMW █

Chuẩn bị thư mục cert để cài đặt:

```
adb root
adb shell
mkdir -p /data/local/tmp/cacerts-backup
cp /system/etc/security/cacerts/* /data/local/tmp/cacerts-backup/
mount -t tmpfs tmpfs /system/etc/security/cacerts
cp /data/local/tmp/cacerts-backup/* /system/etc/security/cacerts/
chown root:root /system/etc/security/cacerts/*
chmod 644 /system/etc/security/cacerts/*
chcon u:object_r:system_file:s0 /system/etc/security/cacerts/*
ls /system/etc/security/cacerts/ | wc -l
exit
```

Tính hash filename mà Android yêu cầu

```
openssl x509 -inform DER -in cacert.cer -out cacert.pem
openssl x509 -inform PEM -subject_hash_old -in cacert.pem | head
-1
```

=> ra được 1 mã hash ngắn, lấy mã hash đó, đổi tên và cài cert vào máy (ví dụ ở đây là **9a5ba575**)

```
adb push cacert.pem /sdcard/9a5ba575.0
adb shell "cp /sdcard/9a5ba575.0
/system/etc/security/cacerts/9a5ba575.0"
adb shell "chmod 644 /system/etc/security/cacerts/9a5ba575.0"
adb shell "chown root:root
/system/etc/security/cacerts/9a5ba575.0"
adb shell "chcon u:object_r:system_file:s0
/system/etc/security/cacerts/9a5ba575.0"
```

Xác nhận lại:

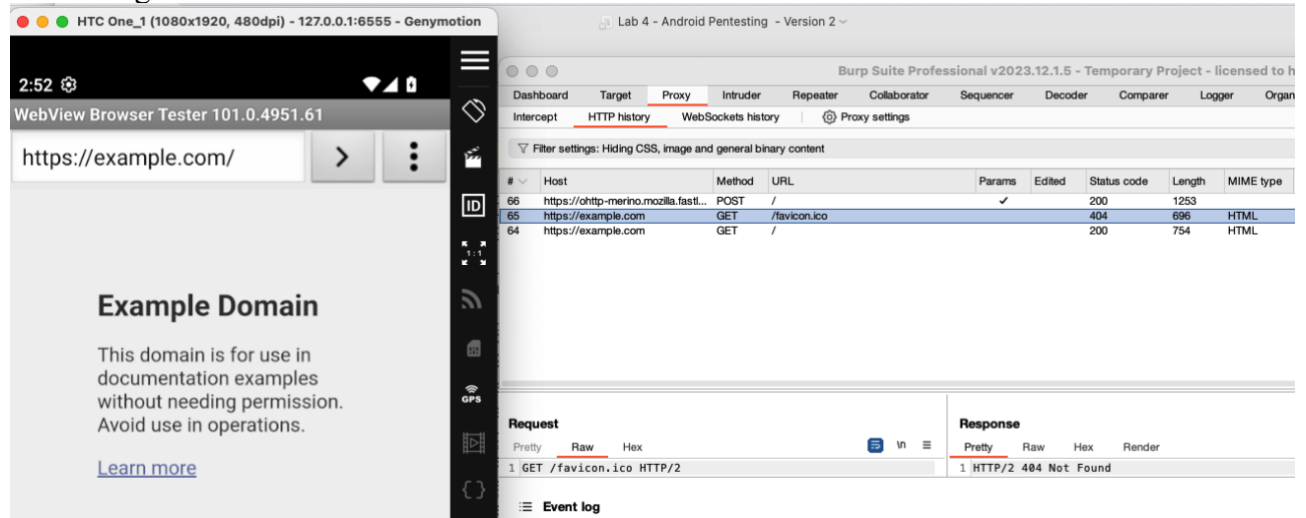
```
adb shell ls -la /system/etc/security/cacerts/ | grep <hash>
adb shell am start -a android.intent.action.VIEW -d
"https://example.com"
```

→ **BMW** adb shell ls -la /system/etc/security/cacerts/ | grep "9a5ba575"

-rw-r--r-- 1 root root 1326 2026-05-13 02:45 9a5ba575.0

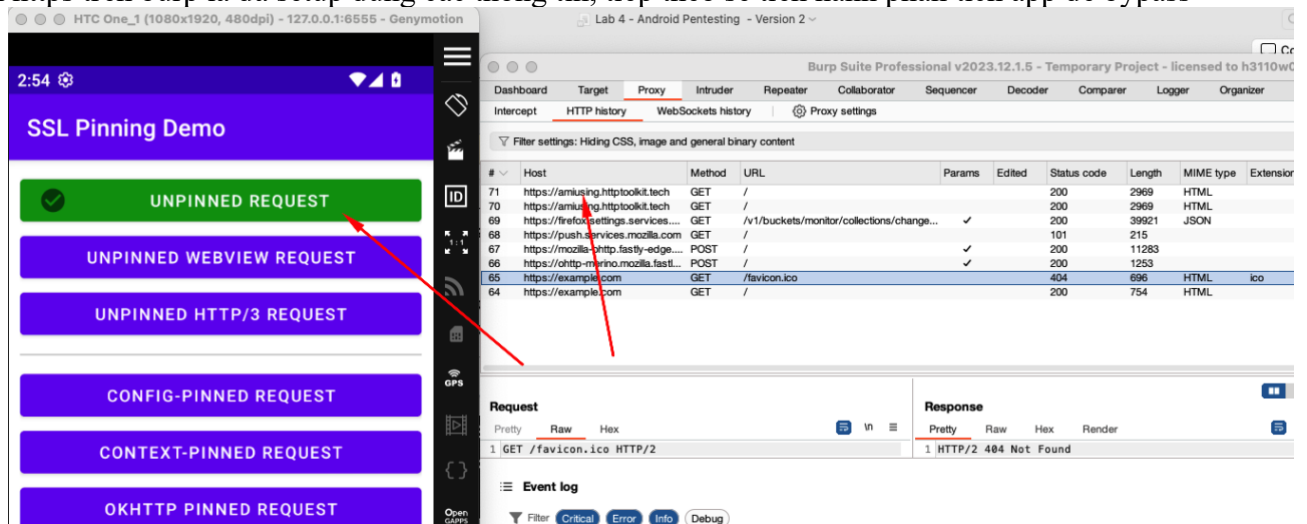
→ **BMW** adb shell am start -a android.intent.action.VIEW -d "https://example.com"
Starting: Intent { act=android.intent.action.VIEW dat=https://example.com/... }

Quan sát trên android thấy có mở trang web và burp thấy có gói tin https đi qua là đã cài cert thành công



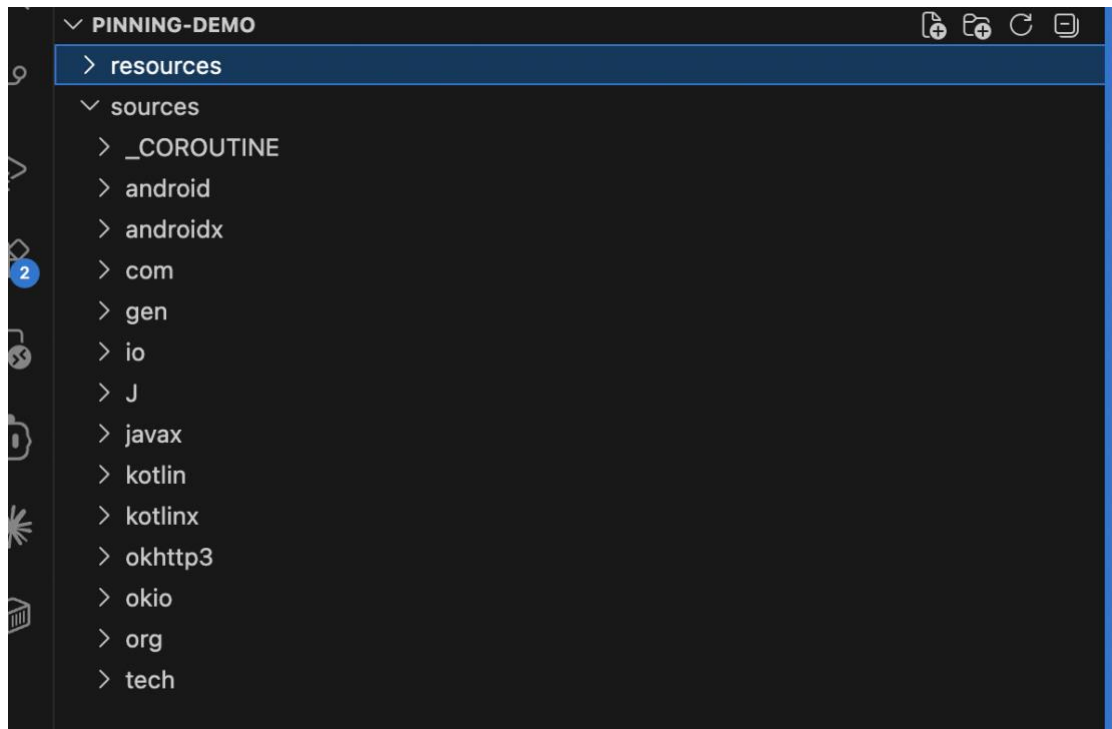
=>>> **lưu ý nếu reboot VM thì các bước trên phải làm lại từ đầu.**

Kiểm tra trên app pinning-demo, khi click vào nút unpinned request sẽ thấy màu xanh và có gói tin https trên burp là đã setup đúng các thông tin, tiếp theo sẽ tiến hành phân tích app để bypass



Trích xuất mã nguồn bằng jadx:

```
jadx pinning-demo.apk
```

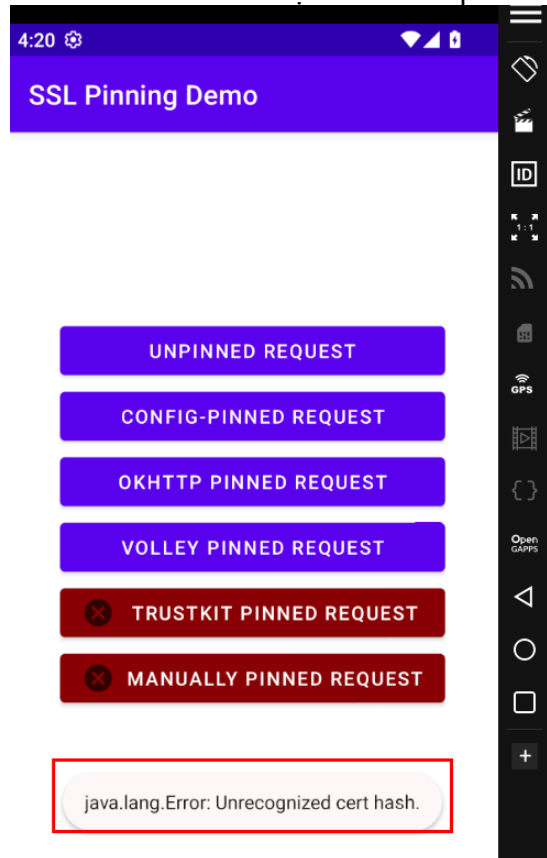
Hình 1 File mã nguồn sau khi trích xuất ra

Tiếp theo cần xác định chính xác hàm nào chịu trách nhiệm kiểm tra chứng chỉ (Certificate pinning)

Dựa vào Log lỗi: "Unrecognized cert hash" => grep để tìm các hàm xử lý trong source

```
(base) ➔ pinning-demo grep -r "Unrecognized cert hash" sources/  
sources/tech/http toolkit/pinning_demo/MainActivity$sendManuallyCustomPinned$1.java:        throw new Error("Unrecognized cert hash.");  
(base) ➔ pinning-demo
```

=> Sau đó tiến hành đọc source để phân tích



Dựa vào adb logcat để tìm ra hàm gây lỗi cert pinning, tương tự tìm được thông tin thì grep xem trong source xử lý chỗ nào và tiến hành phân tích code

```
05-13 04:24:52.160 5650 5668 D EGL_emulation: app_time_stats: avg=719.53ms min=3.24ms max=37322.17ms count=53
05-13 04:24:53.004 5650 5679 I System.out: java.lang.Error: Unrecognized cert hash.
05-13 04:24:53.033 1676 3913 D CoreBackPreview: Window{80d3048 u0 Toast}: Setting back callback OnBackPressedCallbackInfo{mCallback=android.window.IOnBackPressedCallback$
05-13 04:24:53.038 1441 1732 D gralloc_ranchu: gralloc_alloc: Creating ashmem region of size 528384
05-13 04:24:53.039 1441 1732 E android.hardware.graphics.allocator@2.0-service: open_verbose_path:50: Could not open '/dev/goldfish_pipe': No such file or directory
05-13 04:24:53.039 1441 1732 E android.hardware.graphics.allocator@2.0-service: open_verbose:121: both vsock and goldfish_pipe paths failed
05-13 04:24:53.041 1917 2308 E OpenGLRenderer: Unable to match the desired swap behavior.
05-13 04:24:53.042 1441 1732 D gralloc_ranchu: gralloc_alloc: Creating ashmem region of size 528384
05-13 04:24:53.045 1441 1732 E android.hardware.graphics.allocator@2.0-service: open_verbose_path:50: Could not open '/dev/goldfish_pipe': No such file or directory
05-13 04:24:53.045 1441 1732 E android.hardware.graphics.allocator@2.0-service: open_verbose:121: both vsock and goldfish_pipe paths failed
05-13 04:24:53.045 1441 1732 D gralloc_ranchu: gralloc_alloc: Creating ashmem region of size 528384
05-13 04:24:53.047 1441 1732 E android.hardware.graphics.allocator@2.0-service: open_verbose_path:50: Could not open '/dev/goldfish_pipe': No such file or directory
05-13 04:24:53.047 1441 1732 E android.hardware.graphics.allocator@2.0-service: open_verbose:121: both vsock and goldfish_pipe paths failed
05-13 04:24:53.048 1917 2308 W Parcel : Expecting binder but got null!
```

C.4.3.1 Cách 1: Bypass động bằng Frida

Phương pháp này can thiệp trực tiếp vào bộ nhớ của ứng dụng lúc runtime để ghi đè các hàm kiểm tra chứng chỉ SSL. OWASP khuyến nghị sử dụng script frida-multiple-unpinning vì nó hỗ trợ bypass nhiều thư viện mạng khác nhau (như OkHTTP, TrustManager...).

- **Bước 1:** Đảm bảo frida-server đang chạy trên máy ảo Android (đã làm ở phần C.4.1).
- **Bước 2:** Cấu hình Wifi của máy ảo trở Proxy về Burp Suite (đã làm ở C.3.1).
- **Bước 3:** Ở bước này chạy frida để hook và bypass kiểm tra SSL

Yêu cầu 9 Hoàn thành script để hook frida

```
Java.perform(() => {
    console.log("[*] Script attached");

    try {
        const CP = Java.use("okhttp3.CertificatePinner");
        CP.check.overload('java.lang.String',
            'java.util.List').implementation = function (h, c) {
                console.log("[+] OkHttp bypass: " + h);
            };
        console.log("    OK: OkHttp hooked");
    } catch (e) { console.log("    SKIP OkHttp: " + e); }

    try {
        const NSTM =
        Java.use("android.security.net.config.NetworkSecurityTrustManager"
        );
        NSTM.checkPins.implementation = function (c) {
                console.log("[+] NSC bypass");
            };
        console.log("    OK: NSC hooked");
    } catch (e) { console.log("    SKIP NSC: " + e); }

    try {
        const TMI =
        Java.use("com.android.org.conscrypt.TrustManagerImpl");
        TMI.checkTrustedRecursive.implementation = function () {
                console.log("[+] Conscrypt bypass");
                return Java.use("java.util.ArrayList").$new();
            };
    }
});
```

```

        console.log("    OK: Conscrypt hooked");
    } catch (e) { console.log("    SKIP Conscrypt: " + e); }

    try {
        const TK =
Java.use("com.datatheorem.android.trustkit.pinning.PinningTrustMan
ager");
        TK.checkServerTrusted.implementation = function (chain,
authType) {
            console.log("[+] TrustKit bypass");
        };
        console.log("    OK: TrustKit hooked");
    } catch (e) { console.log("    SKIP TrustKit: " + e); }

    try {
        const MA = Java.use("****.MainActivity");
        MA.****.implementation = function (pin, cert) {
            console.log("[+] **** CALLED, returning true");
            return true;
        };
        console.log("    OK: Manually pinned request hooked");
    } catch (e) { console.log("    SKIP MainActivity: " + e); }

    console.log("[*] All hooks done");
});

```

- **Bước 4:** Thực thi Frida để nhúng script vào ứng dụng mục tiêu

Chạy lệnh để hook:

```
frida -U -f tech.httptoolkit.pinning_demo -l bypass.js
```

```

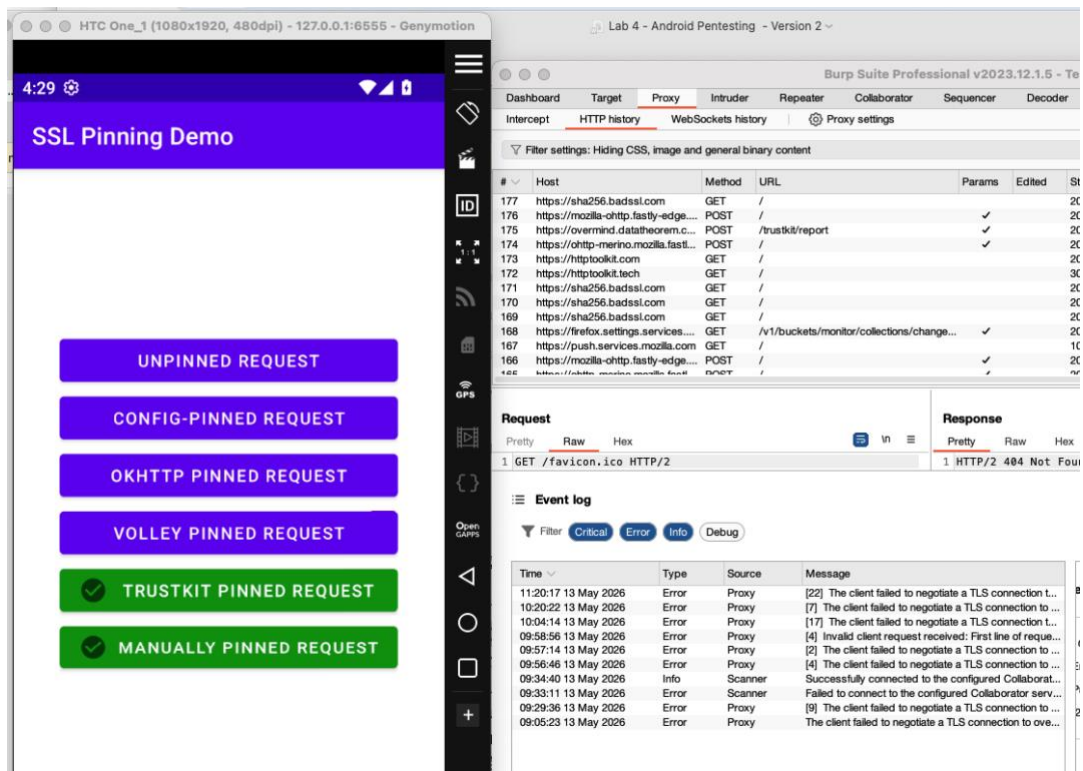
(base) → BTW frida -U -f tech.httptoolkit.pinning_demo -l bypass.js

-----
/ _ |   Frida 17.9.8 - A world-class dynamic instrumentation toolkit
| (| |
> _ |   Commands:
/_/ |_ |   help      -> Displays the help system
. . . .   object?    -> Display information about 'object'
. . . .   exit/quit  -> Exit
. . . .
. . . .   More info at https://frida.re/docs/home/
. . . .
. . . .   Connected to One (id=127.0.0.1:6555)

[One::tech.httptoolkit.pinning_demo ]-> [*] Script attached
OK: OkHttp hooked
OK: NSC hooked
OK: Conscrypt hooked
OK: TrustKit hooked
OK: Manually pinned request hooked
[*] All hooks done

```

- **Bước 5:** Kiểm tra lại trên app, lúc này sẽ thấy được tất cả các pin đã được bypass.
- Yêu cầu 10** trên code trên có 1 pin bị sai, các bạn kiểm tra lại và sửa để chạy cho phù hợp



C.4.3.2 Cách 2: Bypass động bằng Objection

Yêu cầu 11 Bypass tương tự sử dụng Objection

Cài đặt: `pip install objection`

Objection là một công cụ khám phá runtime được xây dựng dựa trên Frida.

- **Bước 1:** Cài đặt Objection qua pip trên máy thật:
- **Bước 2:** Gắn Objection vào tiến trình của ứng dụng mục tiêu:
- **Bước 3:** Tại dấu nhắc lệnh của Objection, gõ lệnh vô hiệu hóa SSL Pinning
- **Bước 4:** Quan sát thông báo trả về. Objection sẽ tự động tìm và hook vào các API liên quan đến Pinning (như TrustManager). Kiểm tra Burp Suite để xác nhận gói tin đã bị bắt.

C.4.3.3 Cách 3: Bypass tĩnh bằng Dịch ngược (Static Patching)

Yêu cầu 12 Tiến hành dịch ngược để bypass manual cert pinning

Nếu ứng dụng sử dụng các thư viện kiểm tra chứng chỉ tùy chỉnh (Custom framework) mà Frida không thể hook tự động, sinh viên phải can thiệp thủ công vào file APK.

- **Bước 1:** Dùng apktool để dịch ngược ứng dụng (như phần C.2.2).
- **Bước 2:** Tiến hành tìm kiếm các điểm ghim chứng chỉ trong mã nguồn smali hoặc thư mục assets:
 - Tìm mã băm chứng chỉ: Gõ lệnh `grep -ri "sha256\|sha1" ./smali`. Sau đó thay thế các mã băm tìm được bằng mã băm chứng chỉ CA của Burp Suite.
 - Tìm file chứng chỉ vật lý: Gõ lệnh `find ./assets -type f \( -iname \*.cer -o -iname \*.crt \)`. Thay thế các file này bằng file chứng chỉ PortSwiggerCA.cer của Burp Suite (đảm bảo đúng định dạng).
 - Tìm Keystore/Truststore: Tìm các file .jks hoặc .bks và đưa chứng chỉ của Burp Suite vào trong kho khóa đó.
- **Bước 3:** Đóng gói (rebuild) lại thành APK mới, thực hiện ký (sign) lại ứng dụng và cài đặt lên máy ảo (như phần C.2.3).
- **Bước 4:** Bật Burp Suite và thao tác trên ứng dụng đã được patch để kiểm tra.

C.5 Khai thác các Component của Android

Khai thác Component (IPC) là đặc sản của Android. Trong phần C.1, đã dùng MobSF và thấy các Activity, Receiver, Provider bị exported=true. Ở phần này sẽ tiến hành phân tích để khai thác thêm

C.5.1 Content Provider

Yêu cầu 13 Trích xuất lịch sử đăng nhập của user vào app insecurebankv2 mà không cần mở app.

Kết quả mẫu:

```
(chase) → Documents adb shell content query --uri content://com.example.insecurebankv2/login
```

```
Row: 0 id=1, name=dinesh
```

C.5.2 Broadcast Receiver

Yêu cầu 14 Dùng lệnh broadcast để kích hoạt sự kiện ngầm trong app insecurebankv2.

```
adb shell am broadcast -a <ACTION> [tùy chọn]
```

C.5.2.1 Bước 1 — Tìm action mà app đang lắng nghe

Có 2 nguồn:

(a) **AndroidManifest.xml** — receiver đăng ký tĩnh:

bash

```
# Trong thư mục đã jadx APK
cat resources/AndroidManifest.xml | grep -B 2 -A 5 "receiver\|action android:name"
```

Tìm pattern:

```
<receiver android:name=".SecretReceiver" android:exported="true">
  <intent-filter>
    <action android:name="com.example.app.AAAA"/>
  </intent-filter>
</receiver>
```

→ Action là `com.example.app.AAAA`.

(b) **Code Java/Kotlin** — receiver đăng ký runtime bằng `registerReceiver()`:

```
grep -rn "registerReceiver\|IntentFilter" sources/ | head -20
```

Tìm pattern:

```
IntentFilter filter = new IntentFilter("com.example.app.AAAA");
context.registerReceiver(receiver, filter);
```

C.5.2.2 Bước 2 — Kiểm tra *exported* của receiver

Từ Android 12 trở đi, `receiver exported="false"` không thể bị trigger từ shell (trừ khi shell có quyền đặc biệt). Trong manifest:

```
<receiver android:name=".SecretReceiver" android:exported="true">
```

Nếu `exported="false"` → bỏ qua, không trigger từ ADB được. Phải dùng cách khác (Frida `sendBroadcast` trong context của app).

C.5.2.3 Bước 3 — Gửi broadcast

Trường hợp đơn giản (action thường, không cần extras):

```
adb shell am broadcast -a com.example.app.AAAA
```

Chỉ định package đích: (không ảnh hưởng receiver khác)

```
adb shell am broadcast -a com.example.app.AAAA -p com.example.app
```

Chỉ định component chính xác:

```
adb shell am broadcast -n com.example.app/.SecretReceiver -a com.example.app.AAAA
```

Các trường hợp kèm extras:

```
adb shell am broadcast -a com.example.app.AAAA \
  --es username "admin" \
  --es password "test123" \
  --ei userid 42 \
  --ez debug true
```

D. CHALLENGES CTF (BÀI TẬP BẢO MẬT ỨNG DỤNG)

D.1 EVABS

Tập tin APK: **EVABsv5.apk**

Định dạng flag: `EVABS{s0m3_rand0m_fl2g_h3r3}`

Challenges 1 Hoàn thành 12 levels.

D.2 Netsight

Hướng dẫn setup challenge

```
adb push bmw.txt /data/data/com.tcpip.netsight/files
```

Flag nằm tại `/data/data/com.tcpip.netsight/files/bmw.txt`

Challenges 2 Tìm cách lấy flag **sử dụng giải pháp sau:** Cài đặt ứng dụng lên máy android để đọc file flag của file apk netsight.apk vừa cài đặt. Các phương pháp lấy flag khác không được accept

* *Viết **writeup** chi tiết*

D.3 Click the Lock

Challenges 3 App yêu cầu click ổ khoá 20k lần để đạt được flag, hãy tìm cách lấy được flag.

* *Viết **writeup** chi tiết*

D.4 r2Pay

Ứng dụng **r2Pay** là một ví điện tử bản debug đang trong giai đoạn pre-release. Đội phát triển tuyên bố ứng dụng có khả năng tạo token thanh toán (gọi là **r2coin**) một cách an toàn về mặt mật mã, và sắp được tích hợp vào nhiều cửa hàng/website.

Flag 1: Tìm cặp PIN + salt sao cho app tạo ra green r2coin.

Định dạng:

r2con{PIN_NUMERIC:SALT_LOWERCASE}

- PIN_NUMERIC: 4 chữ số, ví dụ 1337
- SALT_LOWERCASE: chuỗi salt viết thường, ví dụ radare

Flag 2: Trích xuất **r2Pay master key** ẩn trong tầng whitebox cryptography.

Định dạng:

r2con{ascii(key)}

Trong đó `ascii(key)` là biểu diễn ASCII của key đã trích xuất được.

* *Viết **writeup** chi tiết*

E. YÊU CẦU

- Sinh viên tìm hiểu và thực hành theo hướng dẫn theo nhóm sinh viên đã sắp xếp.
- Nộp mã nguồn (**Code**) và báo cáo kết quả chi tiết những việc (**Report**) đã thực hiện, quan sát thấy và kèm ảnh chụp màn hình kết quả (nếu có); giải thích cho quan sát (nếu có).
- *Đối với mỗi lỗ hổng phát hiện được trong phần C, sinh viên phải phân loại lỗ hổng đó thuộc danh mục nào trong OWASP Mobile Top 10 (Ví dụ: M1, M2, M3...) và giải thích lý do*

Báo cáo:

- File **.PDF**. Tập trung vào nội dung, không mô tả lý thuyết.
- Trong file báo cáo yêu cầu **ghi rõ** nhóm sinh viên thực hiện.
- Đặt tên theo định dạng: [Mã lớp]-Lab2_MSSV1-MSSV2-MSSV3.pdf
Ví dụ: [NT213.K11.ANTN.1]-Lab2_1552xxxx-1552yyyy-1552zzzz.pdf
- Nếu báo cáo có nhiều file, nén tất cả file vào file .ZIP với cùng tên file báo cáo.
- Nộp file báo cáo trên theo thời gian đã thống nhất tại courses.uit.edu.vn.

Đánh giá: Sinh viên hiểu và tự thực hiện được bài thực hành. Khuyến khích:

- Chuẩn bị tốt và đóng góp tích cực tại lớp.
- Có nội dung mở rộng, ứng dụng trong kịch bản phức tạp hơn, có đóng góp xây dựng bài thực hành.

Bài sao chép, trể,... sẽ được xử lý tùy mức độ vi phạm.

HẾT